

FoldingAgent: Inferring Parametric Origami Procedures from Demonstration Videos

MAYA MORIYA, Weizmann Institute of Science, Israel

SIGAL RAAB, Weizmann Institute of Science, Israel

Yael VINKER, MIT, USA

TALI DEKEL, Weizmann Institute of Science, Israel

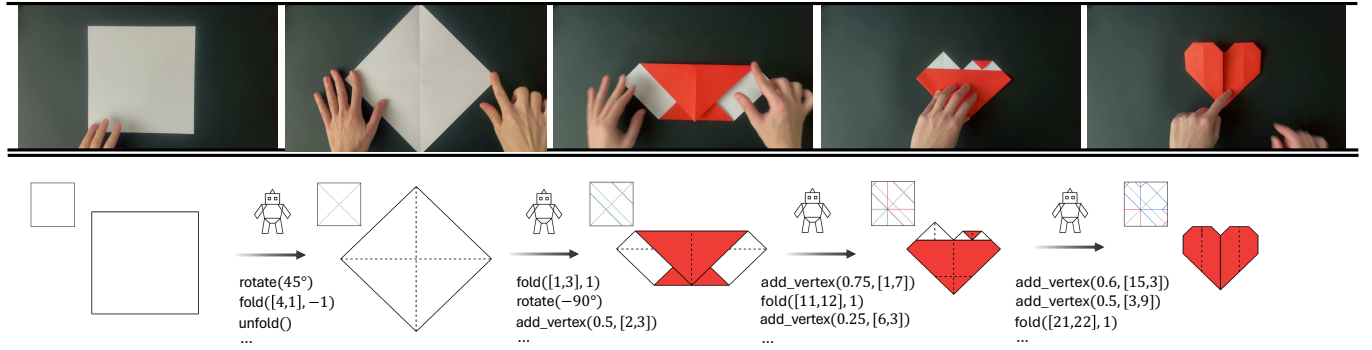


Fig. 1. Given a sequence of keyframes from an instructional origami video (top), our framework, FoldingAgent, infers the full folding procedure as a sequence of executable simulator actions. For each transition, we illustrate the current estimated paper geometry, the predicted simulator actions used to obtain the next paper state, and the accumulated crease pattern. The recovered program is parametric and structured, supporting re-rendering, editing, and analysis.

We present *FoldingAgent*, an agentic framework for inferring explicit parametric folding programs directly from Origami demonstration videos. Our framework leverages the reasoning power of a pre-trained Vision-Language Model (VLM) equipped with a suite of specialized tools that enable the agent to simulate geometric transitions, verify physical plausibility, retrieve and compare visual content, and evaluate its own predictions. To translate visual content into folding programs, we define a parametric space that consists of the paper’s geometry and a set of parametric folding actions. Unlike models that predict static crease patterns, our agent operates sequentially and possesses the ability to re-plan its actions, effectively mitigating the compounding errors inherent in multi-step folding. Our approach takes a step toward closing the gap between human origami knowledge, which is primarily shared through unstructured visual demonstrations, and computational methods, which typically rely on structured, parametric representations such as a crease pattern or an executable parametric plan. We evaluate our approach on PurelandFold, a newly curated benchmark of diverse Pureland origami videos with ground-truth geometry and action labels. Our results demonstrate that by combining VLM reasoning with a set of specialized tools and physical simulation, we can successfully transform unstructured visual demonstrations into executable, physically plausible folding procedures. Project page: <https://maya-moriya.github.io/origami-page>.

1 Introduction

Origami is the art of transforming a flat sheet of paper into a sculpted form through folding. It combines simple materials with rich expressiveness, making it a widely practiced traditional art form. Beyond art, its unique interplay of geometry, structure, and transformation has made it a subject of study across mathematics [Demaine and O’Rourke 2007; Eppstein 2025], engineering [Filipov et al. 2017],

design [Dudte et al. 2021; Lang 1996], and robotics [Balkcom and Mason 2008; Namiki and Yokosawa 2021].

Despite its widespread multidisciplinary applications, a profound disconnect remains between the human practice of origami and its computational counterpart. Origami is fundamentally procedural: a final form emerges from an ordered sequence of folding actions. In practice, this procedure is typically communicated through intuitive visual instructions, such as step-by-step illustrations or video demonstrations. Conversely, computational approaches typically require structured, explicit forms such as a static crease pattern – a compact, order-independent representation of the unfolded paper annotated with all mountain and valley creases required to produce the model. This creates a significant semantic gap between the “language of folding” used by humans and the structured representations required for machine analysis. Addressing this gap is essential for unlocking the vast repository of human origami knowledge for computational use, for example, by enabling robots to learn complex paper-handling skills directly from the thousands of instructional videos available online or by developing origami generative models.

In this work, we take a step toward this goal by posing a new task: translating an origami demonstration video into procedural folding programs that capture the underlying sequence of human folding actions. Specifically, given a sequence of frames, our goal is to infer a parametric geometric representation of the paper that captures its shape and layered structure, together with the action that transforms each state into the next (see Fig. 1).

Inverting video frames into procedural folding programs poses several key challenges. First, the space of folding actions is diverse and highly complex: operations vary in their geometric effect and in

Authors’ Contact Information: Maya Moriya, Weizmann Institute of Science, Rehovot, Israel; Sigal Raab, Weizmann Institute of Science, Rehovot, Israel; Yael Vinker, MIT, Boston, USA; Tali Dekel, Weizmann Institute of Science, Rehovot, Israel.

the number of layers they manipulate, ranging from simple “mountain folds” (a basic crease across the paper) to intricate “petal folds” that involve shifting multiple layers into new, complex arrangements. This diversity raises the question of parametrization: how to effectively map the intuitive action taken by humans into computational operations executed on the paper’s geometry? Even with a defined action space, inferring these actions from video remains challenging: the folder’s hands may often occlude the paper, and as the model progresses, self-occluding layers accumulate and produce a highly complex internal configuration that is not visible from the video alone. Folds also interact through long-range dependencies – a single action can reposition many layers at once and induce new internal creases far from the visible fold line. Thus, each step must be interpreted in light of the full accumulated state of the paper.

We address these challenges via a novel agentic framework that combines the reasoning capabilities of a pre-trained Visual-Language Model (VLM) with a suite of specialized tools that enable the agent to simulate geometric transitions, verify physical plausibility, retrieve and compare visual content, and self-evaluate its own predictions. Rather than attempting to predict the full procedure at once, the agent operates sequentially across a set of keyframes such that its decisions are made in light of the current paper state produced by previous operations.

The scope of our task encompasses Pureland Origami [Smith 1980], a well-studied subset in which every fold is a flat fold along a single crease. Pureland retains substantial expressive range while reducing complexity to a small set of primitive actions: fold, unfold, rotate, and flip. We expose these operations to the agent through a novel parameterized action space, paired with a simulator that executes each operation and maintains the current paper state – its geometry, layer structure, and crease pattern – at every step. This approach allows the agent to move beyond raw pixels and reason about the underlying physical rules of the paper. A key property of our design is allowing the agent to revert and re-plan a given step, thus mitigating the long-range error compounding that follows from the sequential nature of folding. Together, these components turn an unstructured origami video into an executable procedure, producing a sequence of actions, intermediate graphs, and a final crease pattern. Our evaluation over a diverse set of folding sequences demonstrates the power of our agentic approach and the dramatic gain over vanilla VLM inference.

To summarize, our contributions are as follows:

- Introduced the task of inferring procedural folding programs directly from origami demonstration sequences.
- Designed a new parametric action space and a simulator for procedural Pureland origami.
- Developed a new agentic framework that combines a state-of-the-art VLM with a suite of specialized tools, enabling self-verification and replanning, as well as physical verification.
- Curated PurelandFold, a dataset containing diverse Pureland origami videos with labeled geometry and folding actions.

2 Related Work

2.1 Computational Origami

Geometry-based Computation. Most computational origami methods operate under the assumption that the relevant geometry is already available in an explicit, structured form, such as a crease pattern or a folded-state graph. This assumption underlies works in various areas, including:

- *Analysis & Simulation:* A vast body of work focuses on flat- and rigid-foldability, determining whether a given crease pattern admits a valid folded configuration [Akitaya et al. 2020; Bern and Hayes 1996; Feng et al. 2020; Tachi and Hull 2017]. Similarly, simulators and mechanical models operate on these geometric representations to predict physical behavior and stress [Filipov et al. 2017; Hu and Liang 2020; Ku 2022; Mitani 2007; Tachi 2009; Yasuda et al. 2020].
- *Design & Synthesis:* Design methods typically start from a given geometry and synthesize new origami objects or folding procedures. This includes classical and inverse origami design [Dudte et al. 2021; Lang 1996; Tachi 2010; Zhu and Filipov 2022], interactive design [Ghassaei et al. 2018], and folding-sequence generation from a given geometry [Agarwal et al. 2026; Akitaya et al. 2013; Huang et al. 2026].
- *Fabrication & Robotics:* In engineering, structured representations support downstream fabrication and deployment in 3D/4D printing [Mao et al. 2015; Narumi et al. 2023; Sundaram et al. 2017], industrial sheet-metal folding [Ablat and Qattawi 2018; Qattawi et al. 2014], robotic folding [Balkcom and Mason 2008], and self-folding systems [Felton et al. 2014; Hawkes et al. 2010].

While these works demonstrate the power of explicit geometric models, they do not address the fundamental question of how such representations are obtained in practice. Thus, they cannot ingest the unstructured visual demonstrations through which origami knowledge is naturally shared.

Vision-based Reconstruction and Reasoning. This body of work focuses on recovering origami representations from visual observations. Early efforts focused on interpreting clean drill-book illustrations [Shimanuki et al. 2003] or predicting state classes from controlled camera setups [Namiki and Yokosawa 2021; Shimanuki et al. 2012]. Additional works infer line labels or 3D interpretations from origami line drawings [Kanade 1980; Parodi and Torre 1995; Sabbah 1985], provide interactive folding guidance from camera/MR observations [Chen et al. 2023, 2025; Zhu et al. 2010], or estimate physical folding states in robotic/deployable systems [Lal et al. 2023; Namiki and Yokosawa 2021; Ray et al. 2024]. More recently, Kato et al. [2025] have predicted crease lines from before/after image pairs. However, their approach is limited to isolated action vocabulary in a controlled setting, and does not support the broader Pureland action vocabulary used in our task (e.g., unfold, rotate, or flip). Unlike existing works, which primarily focus on state classification or single-step crease prediction, our method is the first to translate unstructured video keyframes into executable procedural programs. By providing an automated, closed-loop system to convert human-oriented demonstrations into formal representations, we take a step

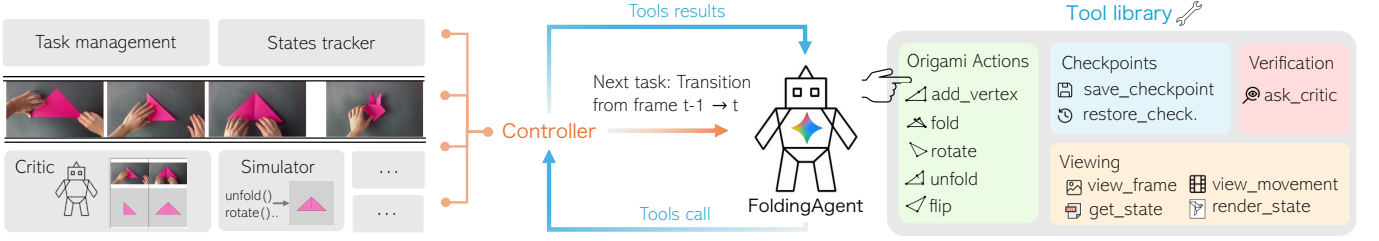


Fig. 2. *FoldingAgent Pipeline*. At each step, the folding agent (a pretrained VLM) infers the action transition between two frames. The agent has the flexibility to issue calls to different tools from the Tool Library (right) – proposing candidate actions, verifying them, revising past decisions, and more. The Controller (orange) holds the agent’s action-prediction timeline, the state history, the input video frames, and can call the origami simulator and the critic (a separate VLM). Tool calls are dispatched by the Controller, with results returned to the agent for further reasoning (blue loop). The agent continues until it commits a checkpoint for frame t , after which the next transition begins.

toward bridging the gap between geometric modeling and real-world origami applications.

2.2 VLM-reasoning for Inverse Vision Tasks

As the reasoning capabilities of VLMs become more powerful, inverse vision is increasingly recast as a language-mediated, agentic process. IG-LLM [2024] demonstrated that VLMs can decode visual embeddings directly into structured 3D programs via spatial reasoning. Recent agentic frameworks follow ReAct-style loops [Yao et al. 2023] to interleave reasoning with action; for instance, VIGA [2026] and IR3D-Bench [2025] employ VLM agents to iteratively write, render, and revise graphics programs based on visual feedback.

This reasoning-centric approach allows VLMs and LLMs to produce editable visual artifacts through executable scripts, e.g. by using Blender [Lu et al. 2025], SVG [Cai et al. 2024; Vinker et al. 2025], Python [Han et al. 2023], Processing [Sharma et al. 2024], or TikZ [Bubeck et al. 2023]. Finally, VLMs serve as powerful critics and reward models for 3D generation. Bai et al. [2025] and Chen et al. [2026] utilize VLM spatial-relation scores to guide text-to-3D optimization. Furthermore, GPT-4V [2024] and Gen3DEval [2025] assess generated assets using human-aligned VLM evaluators, providing a scalable alternative to manual benchmarking.

To the best of our knowledge, Learn2Fold [Huang et al. 2026] is the only prior work to apply LLMs to an inverse origami task. However, it targets a fundamentally different setting: generating folding sequences from a given final crease pattern, which is a structured representation rarely available in real-world origami settings. In contrast, our framework operates directly on instructional origami videos, without requiring additional parametric information, tackling the reconstruction task in a zero-shot agentic fashion.

3 Preliminaries

3.1 FOLD Representation

Flexible Origami List Datastructure (FOLD) is a JSON-based format designed for computational origami models [Demaine et al. 2016]. FOLD has emerged as a standard interchange format, widely adopted across various origami design tools and simulators [Ghasaei et al. 2018; Kraft 2016; Mitani 2007; Tachi 2010]. Like standard mesh formats (OBJ, DXF, etc.), FOLD describes a surface as a set of vertices, edges, and faces, and a per-edge label specifying its

role in the fold — mountain, valley, boundary, or flat. While standard mesh representations are sufficient for general 3D geometry, they are inadequate for flat-folded origami, where overlapping faces stack in a specific order that is part of the folded state. FOLD addresses this with a dedicated field (`faceOrders`) encoding pairwise above/below relationships between faces, allowing it to fully specify flat-folded configurations. In our work, we adopt and extend the FOLD representation (see Sec. 4.1).

4 Method

Given an instructional origami video, we first manually extract a sequence of keyframes $\{I_1, \dots, I_K\}$, where each I_t depicts a meaningful interaction between the demonstrator’s hands and the paper sheet. Our goal is to recover a parametric, executable representation of the demonstrated folding procedure. We formulate the folding procedure as a set of states $\{S_1, \dots, S_K\}$, representing the paper’s geometry at matching keyframes (Sec. 4.1), and corresponding transition actions $\{a_2, \dots, a_K\}$ matching the Pureland action space (Sec. 4.2). We define a novel simulator, detailed in Appendix B.3, that executes an action (or a short sequence of actions) to transform one state into another:

$$\text{Origami-Sim}(S_{t-1}, a_{t-1}) = S_t \quad (1)$$

This formulation enables us to build an automated framework in which a pretrained vision-language model (the *agent*) interacts with the simulator through a dedicated tool library. The agent can propose folding actions, use verification tools to test hypotheses, request additional visual information, and revert earlier decisions, until it reaches a consistent reconstruction. The process is performed sequentially, with rollback available throughout to prevent cumulative errors. A high-level illustration is shown in Fig. 2.

4.1 Origami State Representation

The scope of our task encompasses 2D Pureland origami, which provides a compact and tractable representation space while remaining sufficiently expressive to cover a wide range of real-world instructional folding sequences. Specifically, it restricts the allowed folds to simple mountain and valley folds along straight creases, together with the basic operations unfold, rotate, and flip. Pureland

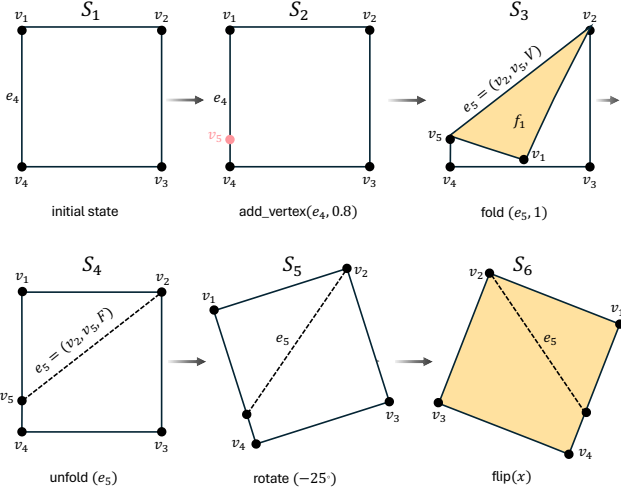


Fig. 3. *State representation and action space.* Starting from a flat square S_1 , we apply the five primitives in sequence (add_vertex, fold, unfold, rotate, flip) and show the states they produce. New vertices, edges, and faces are introduced as the graph evolves; yellow faces are back-side up. Active folds are shown solid, flat creases dashed.

models are accessible to beginners and ensure that every intermediate folded state is a flat-folded layout of polygonal faces, which simplifies both representation and simulation. This setting has been studied in the context of origami notation and robotics [Balkcom 2004; Konjevod and Kuprešanin 2009], and adopted in instructional resources [Smith 1980, 1989, 1993].

Each folded state S is defined as a planar graph as follows:

$$S = (V, F, E, O, L), \quad (2)$$

where V , F , and E are the standard FOLD vertices, faces, and edges, respectively (Sec. 3). Vertices are defined by 2D coordinates, while edges are defined by their endpoint vertices and carry labels in $\{B, F, M, V\}$ (boundary, flat, mountain, and valley). Boundary edges define the outer paper contour, and mountain and valley edges define active folds – creasing the paper so the ridge points up or sinks down, respectively. Finally, flat edges connect coplanar faces without a visible crease. Faces are defined by their boundary vertices, and the complete representation is illustrated in Fig. 3.

O and L are two extensions required for our visual task. $O = \{f \rightarrow o_f\}$ is the per-face orientation, where $o_f \in \{0, 1\}$ indicates whether, at face f , the front (0) or back (1) side of the paper is facing up. Differentiating the paper’s front and back sides improves the VLM’s visual state tracking. $L = [L_1, \dots, L_M]$ is an explicit layer ordering over coplanar faces, listed bottom to top. Each L_i contains several planes, all located at the same topological depth in the folded model; each plane comprises a maximal set of faces connected by flat edges. The faces within each plane move together under any subsequent fold. Operating at the granularity of planes makes layer recomputation tractable after a fold and plays the role of the faceOrders field in FOLD. Together, these fields form a compact representation of a flat-folded pureland origami state, shared by our

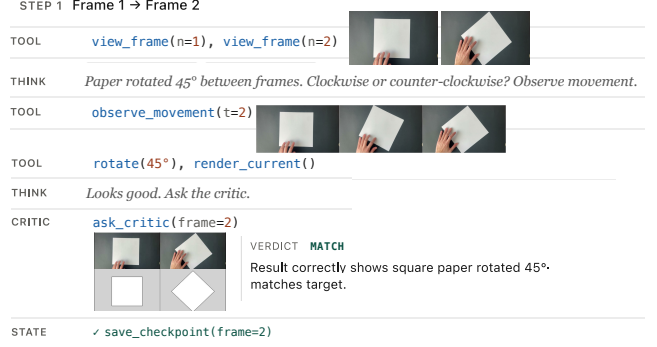


Fig. 4. *Example agent trajectory for a single transition.* A real log showing the agent interleaving reasoning (italic), tool calls, and verification. Here the agent rotates the paper by 45° and confirms the result via the critic, which receives source/target photos (A, B) and rendered states (C, D).

simulator and agent. Note that our geometric representation can be straightforwardly converted to the crease pattern (CP) format.

4.2 Folding Action Space

We define a compact, composable action space to express the pureland folding space. An action a_t is drawn from five primitives:

- add_vertex(e, p)** subdivides edge e at fractional position $p \in [0, 1]$, returning a new vertex. This enables folds along creases that do not pass through existing vertices.
- fold(e, d)** folds the paper along edge e , with direction $d \in \{+1, -1\}$ specifying which side of the crease moves toward the viewer.
- unfold(e)** reverses a previously applied fold along e , leaving a flat edge as a crease mark.
- rotate(θ)** rotates the entire model by θ degrees in the plane.
- flip(α)** reflects the model about an axis $\alpha \in \{x, y, y=x, y=-x\}$.

Fig. 3 illustrates the actions and their cumulative effect on the rendered states. Note that a single transition $I_{t-1} \rightarrow I_t$ may require composing several primitives: for example, folding along a non-vertex crease entails an add_vertex followed by a fold, and reorienting the paper before a fold may require a rotate or flip. By keeping the primitives minimal and composable, the agent can express a wide range of folds with a small vocabulary, while each individual action has an easy-to-verify effect on the state.

4.3 Agentic Framework

We use a pretrained vision-language model as our *agent*, in a zero-shot manner without any fine-tuning. Starting from a square paper S_1 , the agent’s task is to infer a_t for each transition $I_{t-1} \rightarrow I_t$. A representative single-transition trajectory is shown in Fig. 4.

Our system supports the agent through three main components, as illustrated in Fig. 2:

Tool Library. We design four categories of specialized tools:

- **Origami Actions.** The folding primitives in Sec. 4.2 (add_vertex, fold, unfold, rotate, flip) are exposed as callable tools. The simulator applies each tool to the input state and returns the updated state.

- **Viewing.** The agent can inspect both the input sequence and its own progress. `view_frame( $t$ )` retrieves frame I_t and feeds it to the agent; `observe_movement( $t$ )` returns a film-strip of the motion between I_{t-1} and I_t , used when a single keyframe is insufficient to disambiguate the action; `get_current_state` returns the current state S as JSON; and `render_current` returns a 2D diagram of S that visually mirrors the input frames (front/back-side coloring, solid edges for active folds, dashed edges for crease marks). The diagram is the agent’s primary visual feedback signal.
- **Checkpoints.** `save_checkpoint( $t$ )` records the current actions and state as the solution for keyframe I_t , and `restore_checkpoint( $t$ )` reverts the simulator to the state saved at t . Checkpoints serve both as the final output of the system and as recovery points during exploration.
- **Verification.** `ask_critic` invokes a separate visual critic (described below), and `generate_checkpoint_overview` produces a side-by-side panel of all saved checkpoints against their target photos, to identify where reconstruction drifted.

Controller. The controller (marked in orange in Fig. 2) is the deterministic execution layer between the agent and the rest of the system. It parses each tool call emitted by the agent, dispatches it to the appropriate backend (simulator for actions, renderer for diagrams, critic for verification), maintains the simulator state across calls, and returns structured results to the agent. The agent does not manipulate states directly; all changes flow through the controller.

Visual Critic. A central design choice in our framework is to delegate verification to a dedicated vision-language model rather than to the agent itself. When the agent believes a transition is complete, it calls `ask_critic`, which assembles a four-image grid: the real photos of the source and target keyframes (I_{t-1}, I_t), and the rendered diagrams of the corresponding simulator states (S_{t-1}, S_t). The critic compares the candidate diagram against the target photo at the level of geometric structure while tolerating the stylistic gap between photo and diagram. The critic returns one of three verdicts — MATCH, MISMATCH, or EXTREME DIVERGENCE — together with a written analysis and a list of discrepancies. Separating the proposer from the verifier in this way reduces the risk of the agent rationalizing its own mistakes.

Reconstruction loop. With these components in hand, the agent operates sequentially. For a transition $S_{t-1} \rightarrow S_t$, the agent is guided via our system prompt to view I_{t-1} and I_t (and optionally view filmstrips, or future/past frames), proposes one or more simulation actions, inspects the resulting diagram, and calls the critic. The agent has the flexibility to make other tool calls and generate several predictions before deciding to call the critic. The agent is instructed, but not required, to follow this protocol: on a MATCH verdict, it should save S_t as a checkpoint, after which the controller advances to the next step; on a MISMATCH, it should attempt the transition again, possibly using a different sequence of actions and with the option to roll back to previous steps. If the agent fails to obtain a MATCH after three attempts on the same transition, it should invoke `generate_checkpoint_overview` to inspect the full reconstruction so far, identify the earliest checkpoint that diverges from its target

photo, and roll back to a confident earlier state before attempting again under different assumptions. This rollback protocol is essential: errors in early transitions propagate, and providing the agent with an explicit mechanism to detect and recover from them is what makes long sequences tractable. See the Appendix for our full protocol.

Bounded exploration. To prevent excessive rollbacks, we impose a budget of C transition attempts per keyframe, where an attempt is any simulator action applied to the input state, regardless of whether the resulting state is kept (and later revisited) or discarded. Throughout the reconstruction process, the controller records each explored keyframe-transition trajectory as a sequence of actions and resulting states. If the agent consumes its budget for a keyframe without obtaining a Match verdict, the controller renders all explored states for that keyframe and invokes a lightweight selector agent (see Appendix B.5), which chooses the state that best matches the target. The selected state, as well as all ancestor states along the selected path, are saved as checkpoints. Importantly, this commitment is final: once the selector agent is invoked for keyframe t , `restore_checkpoint` to keyframe t or any preceding keyframe is disabled, preventing the agent from revisiting the resolved portion of the sequence. This mechanism trades a small amount of local optimality for the guarantee that exploration remains tractable. We use $C = 5$ in our experiments.

5 Results

We evaluate our framework on a collection of instructional pureland origami videos spanning a range of common folding procedures (described in Sec. 5.2.1). We use Gemini 3.1 Pro Preview for our agent and critic models. Full implementation details, including our system prompts, are included in the Appendix, along with extensive visualizations of our results, metric details, decisions, tool calls, and reasoning across full sequences.

On average, per sequence, FoldingAgent issues 140 queries, consuming 4M input and 68K output tokens, costing \$8.9. This reflects the cumulative cost of iterative tool calling and self-correction across a full folding sequence.

5.1 Qualitative Results

We first present selected qualitative results in Figs. 1, 5 and 6, showing rendered folding procedures extracted from input keyframe sequences across diverse concepts. Our method handles challenging occlusions and lighting conditions, as well as complex configurations with accumulating layers. For example, Fig. 5 shows long sequences of 13 and 14 keyframes that include diverse flips and unfolds, as well as vertex definitions requiring precise geometric reasoning, such as in keyframe 9 (top). In addition, the bottom sequence in Fig. 6 contains challenging hand occlusions in keyframes 3, 6, 8 and 9, which our method successfully overcomes.

In Fig. 7, we illustrate the reasoning process of our agent across selected steps within a single sequence, demonstrating its interaction with the controller, use of different tools, and autonomous decision-making process. Specifically, as seen in the transition from keyframe 5 to 6, the agent first predicts an inaccurate vertex location, which the critic approves, resulting in a wrong fold. This error

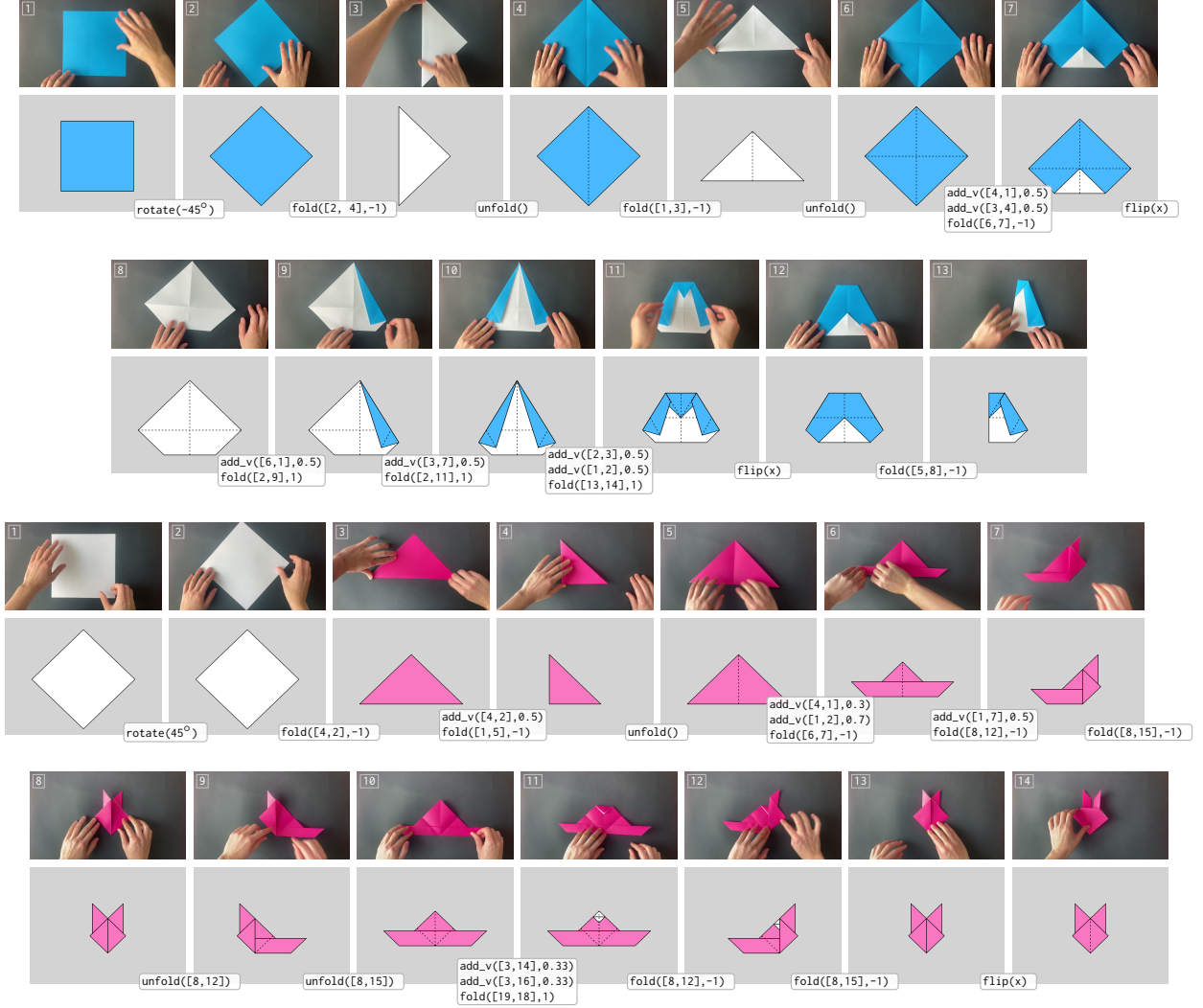


Fig. 5. *Long-sequence qualitative result.* For each example, we show the input keyframes (top), rendering of folded states reconstructed by our method (bottom), and the inferred actions between consecutive keyframes. These examples illustrate our method’s ability to recover a long folding procedure involving rotations, folds, unfolds, flips, and precise vertex definitions. See Appendix A for the full set of results.

is then propagated from keyframe 6 to 7, with both steps approved by the critic. Such failures are possible because the critic has an 88% success rate. However, the agent can recover from these mistakes through our framework. In the transition from keyframe 7 to 8, the agent recognizes that the action involves a flip, but the resulting state is unsatisfactory. It verifies this concern with the critic and then begins investigating the source of the problem by rolling back to keyframe 3. From there, it successfully reconstructs the actions until it reaches the transition from frame 5 to frame 6 again. This time, the agent makes several attempts and autonomously calls tools, such as observing movement between frames, until it reaches a better decision with a more accurate fold. This improved decision then persists through the final fold.

5.2 Quantitative Evaluation

5.2.1 PurelandFold Dataset. Existing origami datasets target either static crease patterns or final folded forms [Agarwal et al. 2026; Spencer et al. 2025; Xu et al. 2025], but none supports the evaluation of procedural folding states from real-world demonstrations. We thus curate a new dataset, *PurelandFold*, comprising 27 diverse folding sequences. We sourced the underlying instructions from the *Easy Origami* category on the OrigamiWay website [2023], which provides a clean and diverse collection of 60 origami models, 40 of which are Pureland. We captured all demonstration videos ourselves, and annotated each sequence with keyframe-level labels of ground-truth folding actions and geometric states (see Secs. 4.1 and 4.2), including paper topology, vertex and face geometry, crease

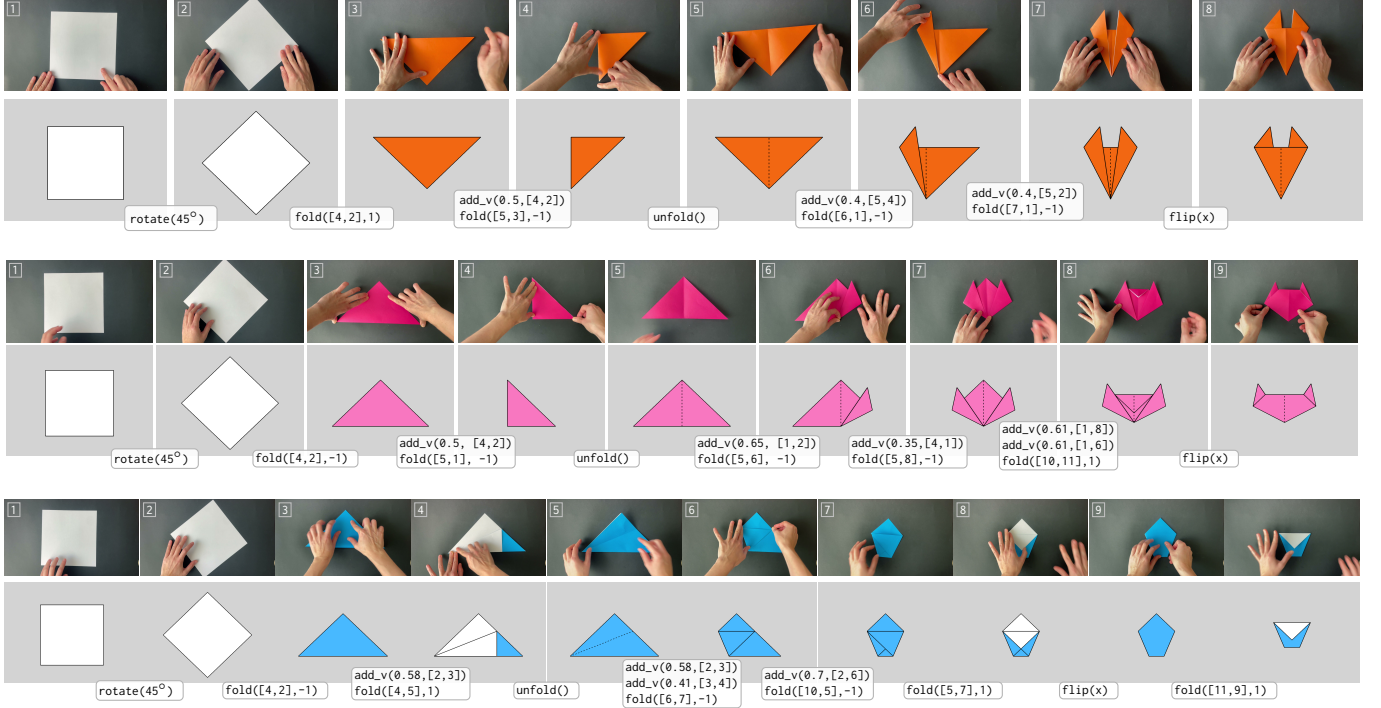


Fig. 6. *Qualitative results.* For each sequence, we show the input keyframes (top) and the rendering of the corresponding folded states predicted by our method (bottom). The actions inferred between consecutive keyframes are shown between the rendered states.

assignments, face orientations, and the layer structure. Using curated instructions and self-capture allows us to achieve an unbiased automatic selection of models, without going into filtering and manual inspection of noisy online videos. Our dataset provides a testbed for evaluating the procedural fidelity of our framework. A link to our dataset is available on our project page.

5.2.2 Metrics. A successful prediction should be (1) *valid* – representing a physically foldable geometry, and (2) *faithful* to the ground truth, both to the annotated geometry and to the raw keyframe. To measure (1), we follow OrigamiSpace [Xu et al. 2025] and define Compilation Validity (CV) as whether the predicted geometry can be processed by the Flat-Folder compiler [Ku 2022]; this requires syntactic validity, consistent topology, and no self-intersections.

Measuring (2) is less straightforward, as faithfulness has several distinct aspects, each of which needs a different metric. We thus follow OrigamiSpace [Xu et al. 2025] and report several complementary metrics: *Topological Structure Similarity (TSS)*, comparing the combinatorial structure of the crease-pattern graph, including vertices, edges, faces, and edge labels. *Geometric Similarity (GS)* evaluates the realized 3D geometry through point-set Hausdorff distance, dihedral angles, and global proportions. *Constraint Satisfaction (CS)* tests whether the prediction satisfies physical folding constraints, including layer ordering and local flat-foldability conditions such as Maekawa’s and Kawasaki’s theorems. *Final Folded State (FFS)* evaluates the overall shape and the layering order. Finally, *Crease-Pattern Dissimilarity (CPD)* [Oh et al. 2015] provides a

comparison via explicit bipartite matching between predicted and ground-truth elements (edges and vertices); unlike OrigamiSpace, which compares patterns by aggregated graph statistics, CPD requires per-element correspondence.

5.2.3 Ablations. To the best of our knowledge, inferring step-by-step folded states from origami videos has not been addressed in prior work, leaving no method that can be directly applied to our task (see the analysis of VIGA [Yin et al. 2026] in Appendix F.2). We thus evaluate against a ladder of variants that progressively add our key components. Unless stated otherwise, all configurations use the same VLM (Gemini 3.1 Pro Preview).

- (1) *VLM-CP*: The VLM is given a pair of frames along with the current crease pattern in standard FOLD representation (I_{t-1}, I_t, CP_{t-1}) , and is prompted to predict CP_t . This variant does not use our representation, tools, simulator, and controller. It serves as a reference for what the underlying VLM can achieve with a conventional crease pattern representation.
- (2) *VLM-S*: Instead of a crease pattern, the VLM works in our representation and action spaces (Secs. 4.1 and 4.2). It receives the current folded state and predicts a symbolic action: $(I_{t-1}, I_t, S_{t-1}) \rightarrow a_t$. This variant isolates the effect of our state and action representation: rather than directly predicting the next geometry, the model predicts an executable simulator action.
- (3) *VLM-S + Critic (VLM-S-C)*: We add the external critic (Sec. 4.3), yet in a *non-agentic* fashion. That is, each step comprises an action prediction followed by critic validation until the critic

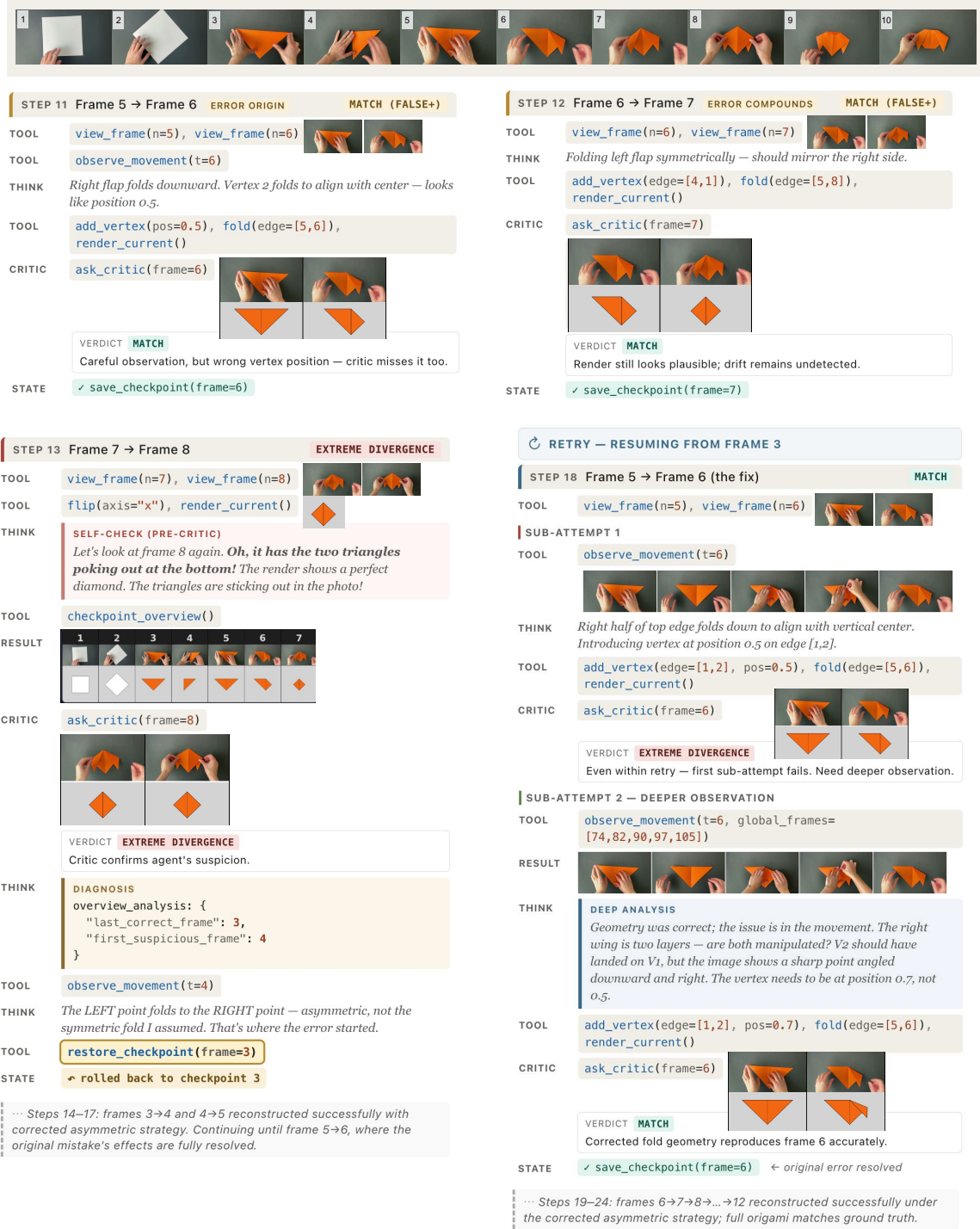


Fig. 7. Recovery from critic-approved errors. The top row shows the full keyframe sequence, while the four panels below illustrate selected reasoning steps. An incorrect fold at frame 5 → 6 is approved by the critic and propagates to frame 7. The agent detects the divergence at frame 7 → 8, rolls back to an earlier checkpoint, and reconstructs the sequence using additional tool calls until it finds a corrected fold that remains consistent through the final frame.

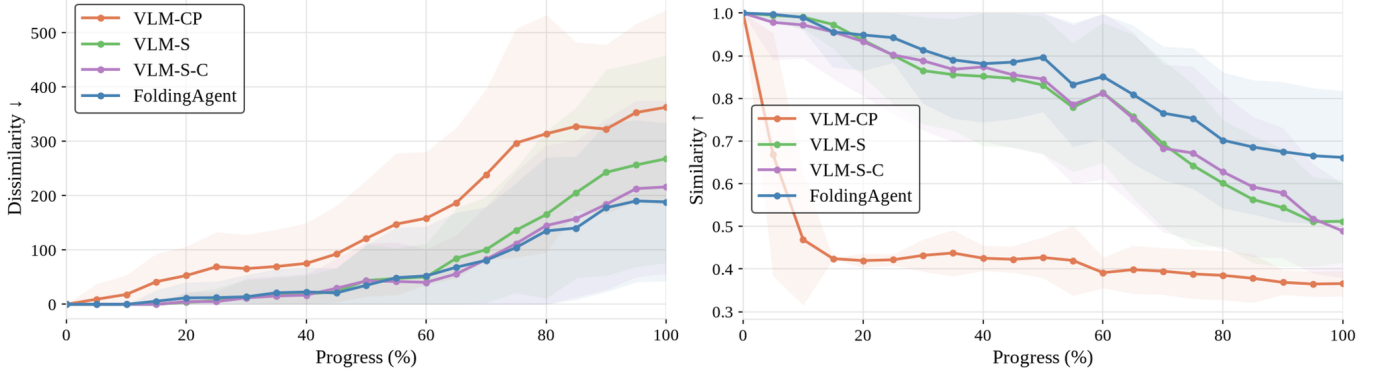


Fig. 8. Performance of our method and ablation variants as a function of sequence progress. We report mean Dissimilarity and Similarity scores averaged over completed sequences. As seen, the performance of all methods declines as the sequence progresses, yet our method exhibits a steadier slope with a notable gap to the others, which increases as the complexity grows.

Table 1. *Quantitative comparison.* We report metrics (Sec. 5.2.2) across all ablation variants (Sec. 5.2.3) and our full framework (FoldingAgent)*. #Comp. denotes the percentage of fully completed sequences and CV the compilation success rate on completed steps; our full method achieves the best scores across all similarity and dissimilarity metrics. We further report the percentage of judgments rating our renderings as closer to the keyframes than those of each variant (VLM-CP could not be evaluated since most of its sequences are not compilable).

Model	#Comp. ↑	CV ↑	Similarity ↑				CPD ↓	User Study
			TSS	GS	CS	FFS		
VLM-CP	96%	8%	0.76 $\pm$ 0.1	0.20 $\pm$ 0.0	0.20 $\pm$ 0.0	0.30 $\pm$ 0.0	363 $\pm$ 179	-
VLM-S	85%	96%	0.79 $\pm$ 0.1	0.51 $\pm$ 0.1	0.53 $\pm$ 0.2	0.21 $\pm$ 0.1	268 $\pm$ 191	84% $\pm$ 5%
VLM-S-C	85%	96%	0.80 $\pm$ 0.1	0.48 $\pm$ 0.1	0.49 $\pm$ 0.2	0.19 $\pm$ 0.1	216 $\pm$ 159	80% $\pm$ 7%
FoldingAgent	100%	96%	0.90$\pm$0.1	0.58$\pm$0.1	0.77$\pm$0.3	0.39$\pm$0.3	189$\pm$145	

*VLM-CP: naive CP prediction; VLM-S: action prediction; VLM-S-C: action + critic.

declares success. If more than 3 attempts are made, the VLM is requested to select the best out of them. This variant allows us to quantify the contribution of richer, more flexible reasoning enabled by our agentic tool-calling procedure.

We evaluate these ablation variants on our dataset. Quantitative results for the final predicted geometry on completed sequences are reported in Tab. 1, and Fig. 8 plots performance across all intermediate steps.

As seen in Tab. 1, applying the VLM without our representation, action space, tools, or agentic framework (VLM-CP) yields invalid geometries in the vast majority of cases, with only 8% of outputs compiling successfully. Shifting the task to our state and action representation (VLM-S) substantially improves compilation rates, quantifying the impact of our representation on producing valid crease patterns, alongside improved topology and geometry scores. Incorporating the critic (VLM-S-C) further improves fine-grained geometry, reflected in a lower dissimilarity score (CPD).

Our full agentic framework improves all metrics significantly, yielding more precise topology, geometry, and crease patterns. This gain stems from the framework’s ability to autonomously detect

errors, roll back incorrect steps, and iteratively self-correct — capabilities absent in the ablated variants. This flexibility does, however, come with a tradeoff: reasoning and tool-calling chains can grow long. We therefore use bounded exploration (Sec. 4.3) to limit the number of attempts per keyframe, and cap execution at 300 total tool calls to constrain runtime and cost.

These differences are further evidenced visually in Fig. 9, where our full method produces notably more accurate final states on completed sequences. We further validate our results through a perceptual user study, using human judgments to assess image-to-render similarity. The participants were shown rendered geometries from two methods (A) and (B), and asked which is more similar to the ground-truth keyframe: A, B, or a tie. We collected 600 judgments over a subset of 10 sequences from our data, comprising all sequences jointly completed by all variants within the tool-call budget. As seen in Tab. 1, our method is strongly preferred over both variants (84% and 80%).

Refer to Appendices E.1 and E.2 for an ablation of in-context examples and backbone generalization results.

6 Conclusion, Limitations, and Future Work

We introduced FoldingAgent, a novel agentic framework capable of translating unstructured origami demonstrations into executable procedural folding programs. By formalizing a parameterized action space for Pureland origami and coupling a pre-trained Vision-Language Model (VLM) with a suite of specialized simulation, verification, and rollback tools, our framework dramatically enhances the VLM’s capabilities in mitigating compounding errors and long-range dependencies inherent in multi-step folding tasks.

As for limitations, our performance is highly dependent on the VLM’s reasoning capabilities. We observed that folds that are heavily occluded — whether by the demonstrator or by complex, accumulating paper layers — remain difficult for the model to accurately identify from raw keyframes. Our agent also struggles to identify compound simultaneous actions (e.g., rotating while flipping the paper), as these break the standard sequential assumptions of our parameterized action space. Finally, our focus in this paper is Pureland

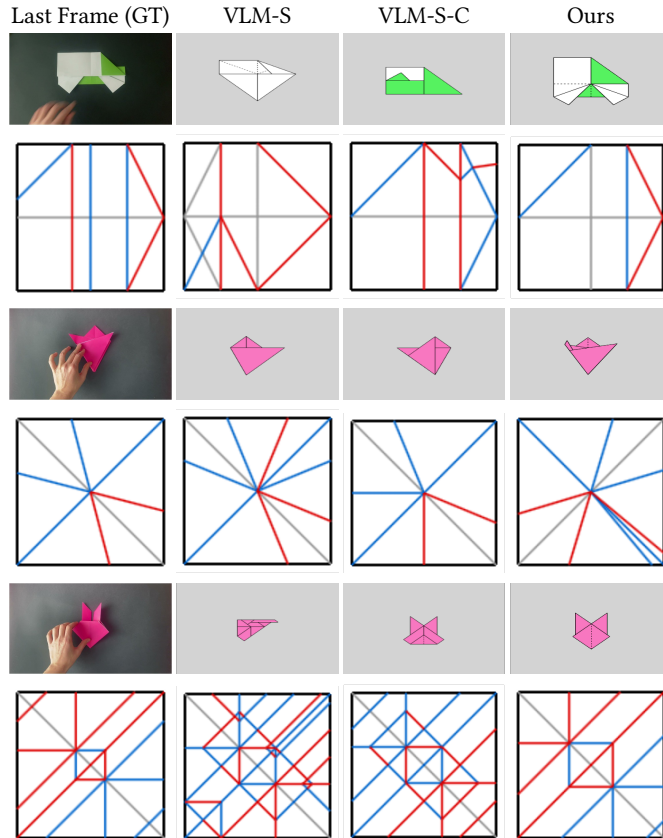


Fig. 9. *Qualitative comparison.* Each row shows a ground-truth keyframe (left) alongside the reconstructed geometry and crease pattern produced by VLM-S, VLM-S-C, and our full framework; mountain and valley creases are marked in red and blue lines, respectively; flat edges are marked in gray.

origami; extending it to more advanced techniques is conceptually possible but requires extending the action space and may require stronger visual-language reasoning (which may not yet be available). We visualize failure cases in Appendix F.1.

Our work takes a first notable step towards bridging the semantic gap between intuitive human visual demonstrations and structured computational origami representations. We believe that the principles we presented for designing a specialized, flexible agentic framework will trigger more research in this area, opening the door to new applications in robotic manipulation, interactive instructional systems, and automated generative design.

Acknowledgments

This research has been generously supported by the Sagol Weizmann-MIT Bridge Program.

References

Muhammad Ali Ablat and Ala Qattawi. 2018. Finite element analysis of origami-based sheet metal folding process. *Journal of Engineering Materials and Technology* 140, 3 (2018), 031008. doi:10.1115/1.4039505

Naaisha Agarwal, Yihan Wu, Yichang Jian, Yikuan Hu, Nishad Mansoor, Mohan Li, Yifei Peng, Wang-Zhou Dai, Yao-Xiang Ding, and Emanuele Sansone. 2026.

OrigamiBench: An Interactive Environment to Synthesize Flat-Foldable Origamis. arXiv:2603.13856 [cs.LG] <https://arxiv.org/abs/2603.13856>

Hugo A. Akitaya, Erik D. Demaine, Takashi Horiyama, Thomas C. Hull, Jason S. Ku, and Tomohiro Tachi. 2020. Rigid Foldability is NP-Hard. *Journal of Computational Geometry (JoCG)* 11, 1 (2020), 93–124. doi:10.20382/jocg.v11i1a4

Hugo A. Akitaya, Jun Mitani, Yoshihiro Kanamori, and Yukio Fukui. 2013. Generating Folding Sequences from Crease Patterns of Flat-Foldable Origami. In *ACM SIGGRAPH 2013 Posters (SIGGRAPH)*. ACM, New York, NY, USA, 1–1. doi:10.1145/2503385.2503407

Weimin Bai, Yubo Li, Weijian Luo, Wenzheng Chen, and He Sun. 2025. Vision-Language Models as Differentiable Semantic and Spatial Rewards for Text-to-3D Generation. arXiv:2509.15772 [cs.CV] <https://ai4scientificimaging.org/vlm3d>

Devin Balkcom. 2004. *Robotic origami folding*. Ph. D. Dissertation. Carnegie Mellon University, Pittsburgh, PA. <https://www.ri.cmu.edu/publications/robotic-origami-folding/>

Devin J. Balkcom and Matthew T. Mason. 2008. Robotic origami folding. *The International Journal of Robotics Research* 27, 5 (2008), 613–627. doi:10.1177/0278364908090235

Marshall Bern and Barry Hayes. 1996. The Complexity of Flat Origami. In *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. ACM/SIAM, New York, NY, USA and Philadelphia, PA, USA, 175–183. <https://dl.acm.org/doi/10.5555/313852.313918>

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 2023. Sparks of Artificial General Intelligence: Early Experiments with GPT-4. arXiv:2303.12712 [cs.CL] <https://arxiv.org/abs/2303.12712>

Mu Cai, Zeyi Huang, Yuheng Li, Haohan Wang, and Yong Jae Lee. 2024. Delving into LLMs’ visual understanding ability using SVG to bridge image and text. <https://openreview.net/forum?id=pwlm6P0e1l>

Qiyu Chen, Richa Mishra, Dina El-Zanfaly, and Kris Kitani. 2023. Origami Sensei: Mixed Reality AI-Assistant for Creative Tasks Using Hands. In *Designing Interactive Systems Conference (DIS)*. ACM, New York, NY, USA, 147–151. doi:10.1145/3563703.3596625

Qiyu Chen, Richa Mishra, Lia Springgarn Purnamasari, Dina El-Zanfaly, and Kris Kitani. 2025. Origami Sensei: A Mixed Reality AI-Assistant. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI)*. ACM, New York, NY, USA, 1–18. doi:10.1145/3706598.3714099

Wenyue Chen, Wenjue Chen, Peng Li, Qinghe Wang, Xu Jia, Heliang Zheng, Rongfei Jia, Yuan Liu, and Ronggang Wang. 2026. Know3D: Prompting 3D Generation with Knowledge from Vision-Language Models. arXiv:2603.22782 [cs.CV] <https://xishuxishu.github.io/Know3D/>

Erik D. Demaine, Jason S. Ku, and Robert J. Lang. 2016. A New File Standard to Represent Folded Structures. <https://github.com/edemaine/fold>. In *Abstracts from the 26th Fall Workshop on Computational Geometry*. CUNY Graduate Center, New York, NY, USA, to appear.

Erik D. Demaine and Joseph O’Rourke. 2007. *Geometric Folding Algorithms: Linkages, Origami, Polyhedra*. Cambridge University Press, Cambridge, UK. doi:10.1017/CBO9780511735172

Levi H. Dudte, Gary P. T. Choi, and L. Mahadevan. 2021. An Additive Algorithm for Origami Design. *Proceedings of the National Academy of Sciences (PNAS)* 118, 21 (2021), e2019241118. doi:10.1073/pnas.2019241118

David Eppstein. 2025. Computational Complexities of Folding. *Journal of Information Processing (JIP)* 33 (2025), 954–973. doi:10.2197/ipsjip.33.954

Samuel M. Felton, Michael T. Tolley, Erik D. Demaine, Daniela Rus, and Robert J. Wood. 2014. A method for building self-folding machines. *Science* 345, 6197 (2014), 644–646. doi:10.1126/science.1252610

Huijuan Feng, Rui Peng, Shixi Zang, Jiayao Ma, and Yan Chen. 2020. Rigid Foldability and Mountain-Valley Crease Assignments of Square-Twist Origami Pattern. *Mechanism and Machine Theory (MMT)* 152 (2020), 103947. doi:10.1016/j.mechmachtheory.2020.103947

Evgueni T. Filipov, Ke Liu, Tomohiro Tachi, Mark Schenk, and Glaucio H. Paulino. 2017. Bar and hinge models for scalable analysis of origami. *International Journal of Solids and Structures* 124 (2017), 26–45. doi:10.1016/j.ijsolstr.2017.05.028

Amanda Ghassaei, Erik D. Demaine, and Neil Gershenfeld. 2018. Fast, interactive origami simulation using GPU computation. In *Origami⁷: Proceedings of the 7th International Meeting on Origami in Science, Mathematics and Education*. Vol. 4. Tarquin, Oxford, England, 1151–1166. https://erikdemaine.org/papers/OrigamiSimulator_Origami7/

Yucheng Han, China. Xiaoyan Zhang, Xin Chen, Xu Yang, Zhibin Wang, Gang Yu, Bin Fu, and Hanwang Zhang. 2023. ChartLlama: A Multimodal LLM for Chart Understanding and Generation. *ArXiv abs/2311.16483* (2023). <https://api.semanticscholar.org/CorpusID:265466206>

Elliot Hawkes, Byoungkwon An, Nadia M. Benbernou, Hiroto Tanaka, Sangbae Kim, Erik D. Demaine, Daniela Rus, and Robert J. Wood. 2010. Programmable matter by folding. *Proceedings of the National Academy of Sciences* 107, 28 (2010), 12441–12445. doi:10.1073/pnas.0914069107

- Yucaí Hu and Haiyi Liang. 2020. Folding simulation of rigid origami with Lagrange multiplier method. *International Journal of Solids and Structures* 202 (2020), 552–561. doi:10.1016/j.ijsolstr.2020.06.016
- Yanjia Huang, Yunuo Chen, Ying Jiang, Jinru Han, Zhengzhong Tu, Yin Yang, and Chenfanfu Jiang. 2026. Learn2Fold: Structured Origami Generation with World Model Planning. arXiv:2603.29585 [cs.GR] <https://arxiv.org/abs/2603.29585>
- Takeo Kanade. 1980. A Theory of Origami World. *Artificial Intelligence (AIJ)* 13, 1–2 (1980), 279–311. <https://www.ri.cmu.edu/publications/a-theory-of-origami-world/>
- Hitomi Kato, Hirotaka Kato, Takatsugu Hirayama, Takahiro Komamizu, and Ichiro Ide. 2025. *Origami Crease Recognition for Automatic Folding Diagrams Generation*. Springer Nature Singapore, Singapore, 16–31. doi:10.1007/978-981-95-4398-4_2
- Goran Konjevod and Ana Maria Kuprešanin. 2009. Notation for a Class of Paperfolded Models. In *Proceedings of Bridges 2009: Mathematics, Music, Art, Architecture, Culture*. Tarquin Group, Banff, Alberta, Canada, 47–54. <https://archive.bridgesmathart.org/2009/bridges2009-47.html>
- Robby Kraft. 2016. Rabbit Ear: Computational Origami Library. <https://github.com/rabbit-ear/rabbit-ear>. Accessed: 2026-05-06.
- Jason S. Ku. 2022. Flat-Folder: A Crease Pattern Solver. <https://github.com/origamimagiro/flat-folder>. GitHub repository.
- Peter Kulits, Haiwen Feng, Weiyang Liu, Victoria Fernández Abrevaya, and Michael J. Black. 2024. Re-Thinking Inverse Graphics With Large Language Models. *Trans. Mach. Learn. Res.* 2024 (2024). <https://api.semanticscholar.org/CorpusID:269302972>
- Rohit Lal, S. Ruphan, C. A. O. Sifan, Sishen Yuan, Lalith, Qiu Liang, and Hongliang Ren. 2023. Unsupervised Intelligent Pose Estimation of Origami-Inspired Deployable Robots. In *Deployable Multimodal Machine Intelligence: Applications in Biomedical Engineering*. Springer, Singapore, 569–589. doi:10.1007/978-981-19-5932-5_21
- Robert J. Lang. 1996. A Computational Algorithm for Origami Design. In *Proceedings of the Twelfth Annual Symposium on Computational Geometry (SoCG)*. ACM Press, New York, NY, USA, 98–105. doi:10.1145/237218.237249
- Parker Liu, Chenxin Li, Zhengxin Li, Yipeng Wu, Wuyang Li, Zhiqin Yang, Zhenyuan Zhang, Yunlong Lin, Sirui Han, and Brandon Y. Feng. 2025. IR3D-Bench: Evaluating Vision-Language Model Scene Understanding as Agentic Inverse Rendering. arXiv:2506.23329 [cs.CV] <https://ir3d-bench.github.io/>
- Sining Lu, Guan Chen, Nam Anh Dinh, Itai Lang, Ari Holtzman, and Rana Hanocka. 2025. LL3M: Large Language 3D Modelers. arXiv:2508.08228 [cs.GR] <https://threedle.github.io/ll3m/>
- Shalini Maiti, Lourdes Agapito, and Filippos Kokkinos. 2025. Gen3DEval: Using vLLMs for Automatic Evaluation of Generated 3D Objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, Los Alamitos, CA, USA, 18552–18562. arXiv:2504.08125 [cs.CV] doi:10.1109/CVPR52734.2025.01729
- Yiqi Mao, Kai Yu, Michael S. Isakov, Jiangtao Wu, Martin L. Dunn, and H. Jerry Qi. 2015. Sequential self-folding structures by 3D printed digital shape memory polymers. *Scientific Reports* 5 (2015), 13616. doi:10.1038/srep13616
- Jun Mitani. 2007. Development of origami pattern editor (ORIPA) and a method for estimating a folded configuration of origami from the crease pattern. *IPSJ Journal* 48, 9 (2007), 3309–3317. <https://ipsj.ixsq.nii.ac.jp/records/9850>
- Akio Namiki and Shuichi Yokosawa. 2021. Origami Folding by Multifingered Hands with Motion Primitives. *Cyborg and Bionic Systems* 2021 (2021). doi:10.34133/2021/9851834
- Koya Narumi, Kazuki Koyama, Kai Suto, Yuta Noma, Hiroki Sato, Tomohiro Tachi, Masaaki Sugimoto, Takeo Igarashi, and Yoshihiro Kawahara. 2023. Inkjet 4D print: Self-folding tessellated origami objects by inkjet UV printing. *ACM Transactions on Graphics* 42, 4 (2023), 1–13. doi:10.1145/3592409
- Seung Man Oh, Godfried T. Toussaint, Erik D. Demaine, and Martin L. Demaine. 2015. A Dissimilarity Measure for Comparing Origami Crease Patterns. In *Proceedings of the International Conference on Pattern Recognition Applications and Methods (ICPRAM)*. SCITEPRESS, Setubal, Portugal, 386–393. doi:10.5220/0005291203860393
- Origami Way. 2023. Origami Instructions – Origami Way. <https://www.origamiway.com/>.
- Pietro Parodi and Vincent Torre. 1995. The Complexity of Understanding Line Drawings of Origami Scenes. *International Journal of Computer Vision (IJCV)* 16 (1995), 139–170. doi:10.1007/BF00055000
- Ala Qattawi, Mahmoud Abdelhamid, Ahmad Mayyas, and Mohammed Omar. 2014. Design analysis for origami-based folded sheet metal parts. *SAE International Journal of Materials and Manufacturing* 7, 2 (2014), 488–498. doi:10.4271/2014-01-9098
- Lala Ray, Daniel Geissler, Bo Zhou, Paul Lukowicz, and Berit Greinke. 2024. Origami Single-End Capacitive Sensing for Continuous Shape Estimation of Morphing Structures. *Scientific Reports (Sci. Rep.)* 14, 1 (2024). doi:10.1038/s41598-024-67149-9
- Daniel Sabbah. 1985. Computing with Connections in Visual Recognition of Origami Objects. *Cognitive Science (Cogn. Sci.)* 9, 1 (1985), 25–50. doi:10.1016/S0364-0213(85)80008-2
- Pratyusha Sharma, Tamar Rott Shaham, Manel Baradad, Stephanie Fu, Adrian Rodriguez-Munoz, Shivam Duggal, Phillip Isola, and Antonio Torralba. 2024. A vision check-up for language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE Computer Society, Los Alamitos, CA, USA, 14410–14419.
- Hiroshi Shimanuki, Jien Kato, and Toyohide Watanabe. 2003. Recognition of Folding Process from Origami Drill Books. In *Proceedings of the Seventh International Conference on Document Analysis and Recognition (ICDAR)*. IEEE Computer Society, Los Alamitos, CA, USA, 550–554. doi:10.1109/ICDAR.2003.1227725
- Hiroshi Shimanuki, Toyohide Watanabe, Koichi Asakura, Hideki Sato, and Taketoshi Ushima. 2012. Folding Support for Beginners Based on State Estimation of Origami. In *2012 IEEE RO-MAN: The 21st IEEE International Symposium on Robot and Human Interactive Communication (RO-MAN)*. IEEE, Piscataway, NJ, USA, 789–794. doi:10.1109/ROMAN.2012.6343855
- John S. Smith. 1980. *Pureland Origami*. Number 14 in BOS Booklet. British Origami Society, UK. 48 pages. <https://www.giladorigami.com/origami-database-book/2388/Pureland-Origami-by-John-Smith>
- John S. Smith. 1989. *Pureland Origami* 2. Number 29 in BOS Booklet. British Origami Society, UK. 72 pages. <https://www.giladorigami.com/origami-database-book/2389/Pureland-Origami-2-by-John-Smith>
- John S. Smith. 1993. *Pureland Origami* 3. Number 43 in BOS Booklet. British Origami Society, UK. 52 pages. <https://www.giladorigami.com/origami-database-book/2390/Pureland-Origami-3-by-John-Smith>
- Ryan Spencer, Roey Yaari, Ritvik Vemavarapu, Joyce Yang, Steven Ngo, and Utkarsh Sharma. 2025. GamiBench: Evaluating Spatial Reasoning and 2D-to-3D Planning Capabilities of MLLMs with Origami Folding Tasks. arXiv:2512.22207 [cs.AI] <https://arxiv.org/abs/2512.22207>
- Subramanian Sundaram, David S. Kim, Marc A. Baldo, Ryan C. Hayward, and Wojciech Matusik. 2017. 3D-printed self-folding electronics. *ACS Applied Materials & Interfaces* 9, 37 (2017), 32290–32298. doi:10.1021/acsami.7b10443
- Tomohiro Tachi. 2009. Simulation of rigid origami. In *Origami 4: Fourth International Meeting of Origami Science, Mathematics, and Education*, Robert J. Lang (Ed.). A K Peters, Natick, MA, 175–187. https://origami.c.u-tokyo.ac.jp/~tachi/cg/SimulationOfRigidOrigami_tachi_4OSME.pdf
- Tomohiro Tachi. 2010. Freeform Variations of Origami. *Journal for Geometry and Graphics* 14, 2 (2010), 203–215. <https://tsg.ne.jp/TT/cg/TachiFreeformOrigami2010.pdf>
- Tomohiro Tachi and Thomas C. Hull. 2017. Self-Foldability of Rigid Origami. *Journal of Mechanisms and Robotics (JMR)* 9, 2 (2017), 021008. doi:10.1115/1.4035558
- Yael Vinker, Tamar Rott Shaham, Kristine Zheng, Alex Zhao, Judith E. Fan, and Antonio Torralba. 2025. SketchAgent: Language-Driven Sequential Sketch Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, Los Alamitos, CA, USA. arXiv:2411.17673 [cs.CV] doi:10.1109/CVPR52734.2025.02175
- Tong Wu, Guandao Yang, Zhibing Li, Kai Zhang, Ziwei Liu, Leonidas Guibas, Dahua Lin, and Gordon Wetzstein. 2024. GPT-4V(ision) is a Human-Aligned Evaluator for Text-to-3D Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, Los Alamitos, CA, USA, 22227–22238. arXiv:2401.04092 [cs.CV] doi:10.1109/CVPR52733.2024.02098
- Rui Xu, Dakuan Lu, Zicheng Zhao, Xiaoyu Tan, Xintao Wang, Siyu Yuan, Jiangjie Chen, and Yinghui Xu. 2025. ORIGAMISPACE: Benchmarking Multimodal LLMs in Multi-Step Spatial Reasoning with Mathematical Constraints. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., Red Hook, NY, USA. NeurIPS 2025 spotlight. <https://openreview.net/forum?id=y7ahj9RoXQ>
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Amherst, MA, USA. arXiv:2210.03629 [cs.CL] https://openreview.net/forum?id=WE_vluYUL-X
- Hiroimi Yasuda, Koshiro Yamaguchi, Yasuhiro Miyazawa, Richard Wiebe, Jordan R. Raney, and Jinkyu Yang. 2020. Data-driven prediction and analysis of chaotic origami dynamics. *Communications Physics* 3 (2020), 168. doi:10.1038/s42005-020-00431-0
- Shaofeng Yin, Jiaxin Ge, Zora Zhiruo Wang, Chenyang Wang, Xiuyu Li, Michael J. Black, Trevor Darrell, Angjoo Kanazawa, and Haiwen Feng. 2026. Vision-as-Inverse-Graphics Agent via Interleaved Multimodal Reasoning. arXiv:2601.11109 [cs.CV] <https://fugtemypt123.github.io/VIGA/>
- Kening Zhu, Owen Noel Newton Fernando, Adrian David Cheok, Mark Fiala, and Theam Wei Yang. 2010. Origami Recognition System Using Natural Feature Tracking. In *2010 9th IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, Piscataway, NJ, USA, 289–290. doi:10.1109/ISMAR.2010.5643611
- Yi Zhu and Evgueni T. Filipov. 2022. Harnessing Interpretable Machine Learning for Holistic Inverse Design of Origami. *Scientific Reports (Sci. Rep.)* 12 (2022). doi:10.1038/s41598-022-23875-6

Appendix

This appendix provides additional details beyond those in the main paper. While the main paper stands on its own, the details given here may shed more light.

Appendix A provides access to the full reconstruction results and agent traces. Implementation and dataset construction are detailed in Appendices B and C, and Appendix D presents the complete prompts. Additional quantitative experiments and metric definitions appear in Appendix E. Finally, Appendices F and G provide additional qualitative results and a visualization of the user study.

A Full Results

All results are available via the *Results* link on our project page: <https://maya-moriya.github.io/origami-page>. It presents a wide range of folding sequences, along with the full agent interaction for each sample. A visualization of the results page and a description of its usage are shown in Fig. 10.

B Implementation Details

Both the main agent and the visual critic are powered by Gemini 3.1 Pro Preview (gemini-3.1-pro-preview). The agent interacts with the controller exclusively through Gemini’s native function-calling API. All tools, such as `fold` and `save_checkpoint`, are declared as typed JSON schemas and dispatched by the controller. This allows the agent to issue multiple tool calls within a single iteration and to chain actions across iterations based on returned results.

Token budget. Each sequence is limited to 300 iterations, with a maximum thinking budget of 8,192 tokens per iteration.

Running time. Across folding sequences, running times range from 9 to 96 minutes, with an average of 31 and a median of 20 minutes.

Thinking-token exhaustion. Although internal thinking improves our framework’s performance, we encountered a developer-reported failure mode in which reasoning tokens can consume the available token budget before a useful response is produced.¹ In such cases, the request terminates with `MAX_TOKENS`, often after producing long, uninformative reasoning traces. We treat excessive thinking-token consumption as a practical failure mode of long agentic runs: when it occurs, we roll back to the beginning of the current folding step and discard the partial actions generated within that step.

B.1 Controller

The controller is implemented as a deterministic execution layer around Gemini’s function-calling interface. In addition to the high-level role described in the main paper, the controller is responsible for validating each emitted tool call against its JSON schema, converting valid calls into backend-specific function invocations, and serializing all returned outputs into the next agent message. Invalid or malformed calls are rejected before reaching the simulator, and

the resulting error message is returned to the agent as part of the interaction history.

Context management. Each folding step begins with a fresh context buffer initialized by a text-only summary of all previous steps. Within a step, the controller stores the full interaction history, including agent responses, tool calls, backend outputs, renders, critic verdicts, and checkpoint operations. At each iteration, this accumulated step history is sent back to the agent together with the result of the most recent tool call, if one exists.

State bookkeeping. The controller keeps a synchronized record of the simulator state, checkpoint table, restore counts, and action history. Checkpoint inspection is read-only, whereas checkpoint restoration replaces the current simulator state and increments a persistent restore counter for that checkpoint. These counters are used for logging and diagnosis; the agent receives them as additional context when deciding whether to continue retrying or roll back further.

Logging and recovery. All interactions are written to a structured log, including the raw function call arguments, backend return values, rendered image paths, critic outputs, and rollback events. When a request terminates early, fails schema validation, or exhausts its token budget, the controller records the failure and restarts from the beginning of the current folding step, preserving only the previously saved checkpoints.

B.2 State and Action Representation

Two details of our representation are specific to reconstruction from visual demonstrations. First, we explicitly model `rotate` and `flip`. These operations are usually irrelevant for purely geometric origami analysis, since they do not change the intrinsic crease pattern, but they are necessary here because the demonstrator may reorient or turn over the paper between video frames.

Second, `fold` is specified by an existing edge rather than by an arbitrary line. The same geometric line can correspond to different folds depending on which faces and layers move. Using an edge anchors the command to the current topology, making clear which faces and layers of the current geometry are affected.

B.3 Simulator

The simulator is a deterministic geometric engine that executes a single action (Sec. 4.2) on a state $S = (V, F, E, O, L)$ and returns the resulting state S' . Recall that V, F, E, O, L denote vertices, faces, edges, orientation, and layering, respectively (Sec. 4.1). We describe the implementation of `fold` in detail below; `unfold` uses similar machinery, and `rotate` and `flip` are straightforward. Fig. 11 demonstrates a simple folding example.

Preliminaries and Notation. Let e be the fold edge and $d \in \{-1, +1\}$ the fold direction argument. e is defined by two vertices in V , not necessarily in the same face. Let ℓ denote the abstract line through e along which the paper is creased. Every edge borders one or two faces; we denote these as the edge’s positive-side face (upper/right) and negative-side (lower/left) face. The direction d defines one of these two sides as the moving side of the fold. We

¹<https://discuss.ai.google.dev/t/thinking-ate-all-the-tokens-and-hit-max-tokens/110971>

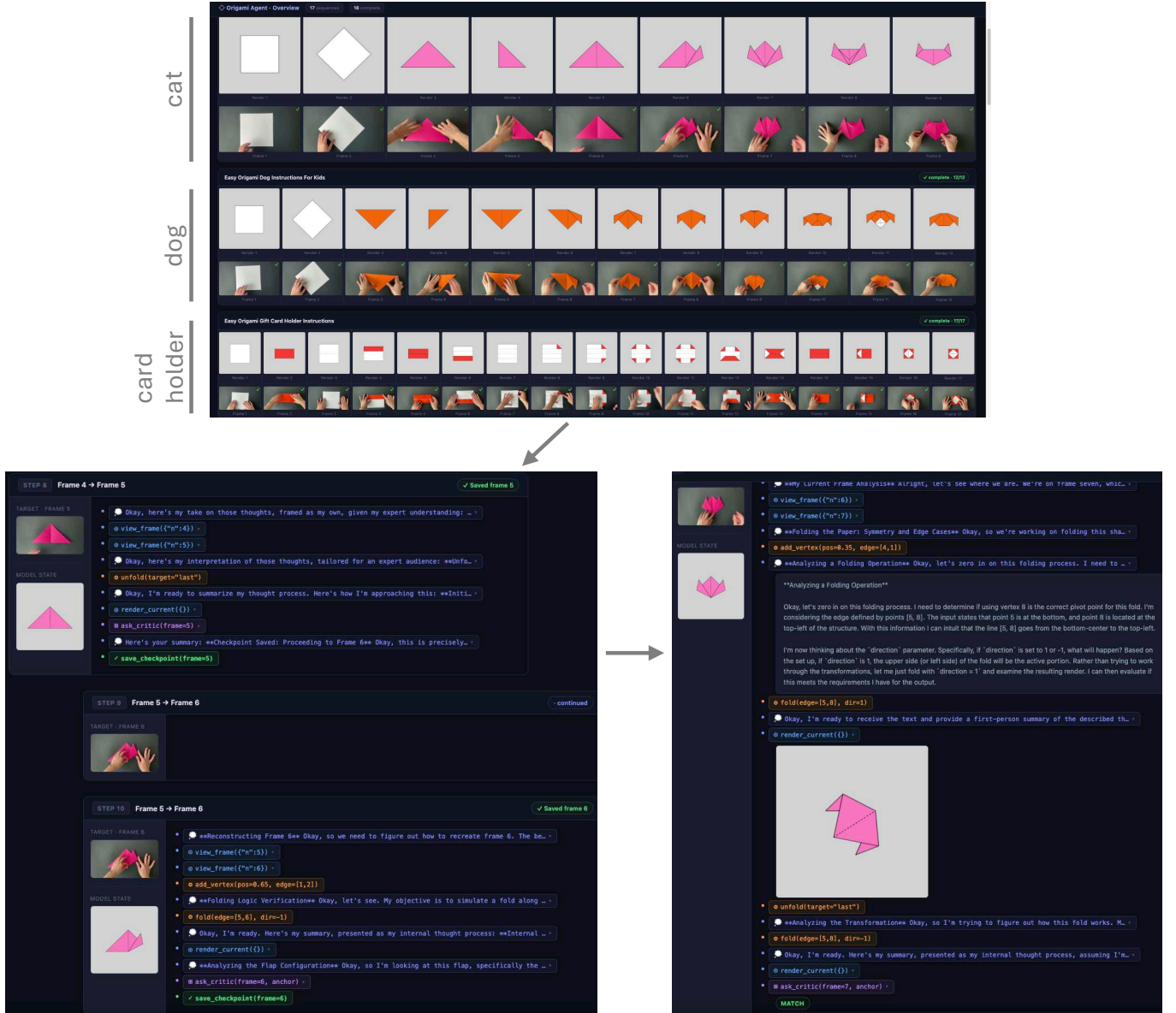


Fig. 10. *Results dashboard*. The main screen (top) shows all reconstructed sequences. Each row corresponds to one folding sequence, with dataset keyframes shown below the rendered geometries. Users can scroll through the sequences to inspect the diversity of examples and the quality of reconstructions. Clicking a sequence opens a detailed trace view (bottom left), which logs all agent interactions. Folding steps are shown with nested indentation, and each interaction element, such as a tool call or response, can be expanded to show details including agent reasoning, requested images or renders, and critic calls and responses. In the example shown (bottom right), the agent’s reasoning and the response to the render_current tool are expanded. After observing the render, the agent recognizes that the current fold is insufficient, continues with additional actions, and eventually reaches the correct fold.

define the *moving set* as the set of planes that will move as part of this fold (built below). A *plane* is a set of faces connected by flat edges. A *layer* is a set of planes that are disjoint and share the same topological depth, i.e., lie above the same set of planes and below

another shared set of planes. We represent L internally as a directed acyclic graph $G_P = (V_P, E_P)$ over planes, where V_P consists of planes, and a relation $p_i \rightarrow p_j$ denotes that plane p_i lies above plane p_j , for

$p_i, p_j \in V_P$. This graph can easily be topologically sorted to a list representation L , which is more interpretable by the agent.

Split faces and planes that straddle ℓ . Each face that straddles ℓ is subdivided into a static sub-face and a moving sub-face; each plane containing both moving and static faces is subdivided into a static sub-plane (keeping its static faces) and a moving sub-plane (keeping its moving faces).

Moving Set. We use the term *moving set* to denote the subset of planes that gets split in the fold. The *moving set* is initialized by the plane(s) containing the endpoints of e . Starting from this set, the simulator repeatedly adds planes to the moving set: any plane that shares an edge lying on the moving side of the fold with a plane already in the moving set; any plane sandwiched between two planes already in the moving set, even if it does not itself share such an edge with them.

Mirroring vertices. Each vertex in the *moving set* is updated so that its new location is the mirror image of its original location across ℓ . See Fig. 11(D).

Assigning plane relations. Planes in the *moving set* are turned over; existing relations in G_P between two moving planes are reversed. In addition, new relations are added to G_P placing moving sub-planes above their former neighbors on the static side of ℓ .

B.4 Critic

Conceptually, the critic performs a relational comparison: it checks whether the simulated transition from source render to candidate render matches the observed transition from source photo to target photo. Thus, the critic does not judge the candidate render in isolation, but evaluates whether the same geometric change is expressed in the photo pair and in the simulator pair. Visual illustrations of such relational comparisons, as well as examples for critic verification, are provided in the main paper.

Interaction with Agent. The agent may invoke `ask_critic` at any point during an iteration, and is explicitly instructed to do so once it believes the current action sequence matches the target frame. The intended control flow is to save a checkpoint after a positive critic verdict and to retry or roll back after a negative one. In practice, however, the agent retains discretion: it sometimes saves a checkpoint without an additional critic call, or decides whether to follow a critic verdict based on the accompanying analysis and its own visual reasoning. This is useful because the critic, while generally reliable, can occasionally produce both false positives and false negatives.

Each call to the critic uses the following inputs.

Images. The critic receives four images: the source photo, target photo, source render, and candidate render. In practice, these images are passed separately; the controller assembles the grid only when returning the critic’s response to the agent.

Action classes. The predicted transition type, such as fold, unfold, rotate, or flip.

Geometry JSON. The source and candidate geometry states.

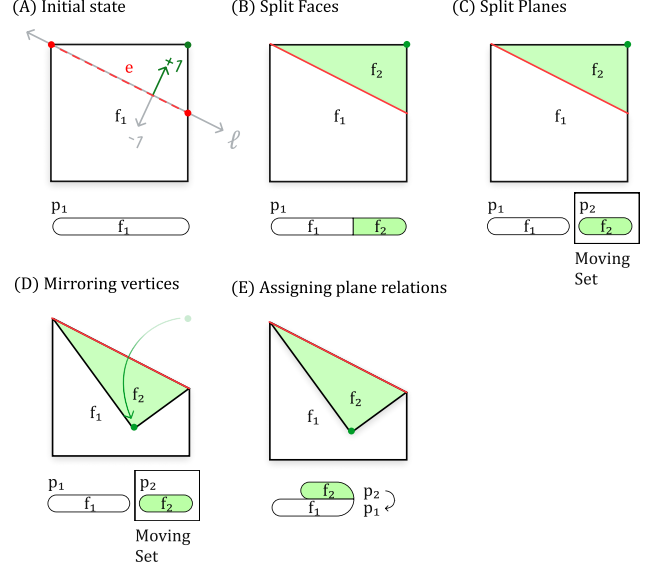


Fig. 11. Detailed run of fold($e, +1$). (A) *Initial state*: a square face f_1 lies in a single plane p_1 . The fold edge e is depicted by the dashed red line, together with the crease line ℓ and its perpendicular directions $+1/-1$. The moving side is colored in green. (B) *Split faces*: f_1 is subdivided along ℓ into a static sub-face f_1 and a moving sub-face f_2 , shown in green. (C) *Split planes*: since f_1 and f_2 originate from the same plane p_1 , the plane is likewise split: f_1 remains on p_1 , while f_2 is assigned to a new plane p_2 , which constitutes the *moving set* for this fold. (D) *Mirroring vertices*: all vertices belonging to the moving sub-plane p_2 are reflected across ℓ . (E) *Assigning plane relations*: the layering graph G_P is updated so that p_2 (containing the folded face f_2) is recorded as lying above p_1 .

Fold anchor. An optional local relation for fold actions, specified by a moving element, a static reference, and their expected relationship. For example, a fold anchor may specify that the upper-left triangular flap is the moving element, the right vertical edge of the base layer is the static reference, and the flap’s free edge should align with that edge while preserving a small visible gap near the top corner.

The critic responds with these details:

Verdict. One of MATCH, MISMATCH, or EXTREME DIVERGENCE.

Analysis. A short textual comparison focused on geometric agreement, including silhouette, flap proportions, fold-line positions, contact topology, gaps, offsets, and layer order.

Anchor check. A sentence indicating whether the fold anchor is satisfied, or that no anchor was provided.

Discrepancies. A list of concrete geometric issues used by the agent to revise the next attempt.

B.5 Selector

The selector is a lightweight VLM-based comparator, assisting the bounded exploration scheme described in the main paper. The selector is shown a target video keyframe, denoted A, together with several candidate diagrams, denoted B1, B2, ..., and selects the candidate whose shape, folds, and proportions best match the target.

Its response is constrained to the label of a single candidate, with no additional text or reasoning. The complete system and user prompts used by the selector are provided at the end.

C Dataset Construction Details

We captured the videos for the PurelandFold dataset using a front-facing LED ring light with a phone holder, together with ambient daylight from a nearby window. Filming was done with an ordinary iPhone 12 device. Folding actions were performed on black Bristol board to provide a high-contrast background. Keyframes representing geometric transitions were manually extracted from each video, and the corresponding actions were annotated. Action classification was usually straightforward, whereas determining numerical action parameters, such as vertex locations, required an iterative trial-and-error process. Altogether, our dataset comprises 27 sequences, with an average length of 12 frames, ranging from 5 to 21 frames.

D Prompts

All prompts are provided in full at the end.

D.1 Agent Prompts

The system prompt provides a comprehensive specification of the agent’s role, environment, available actions, and expected reconstruction behavior. It consists of:

Task definition. Defines the overall goal: reconstructing an origami folding sequence from real-world photos by producing matching simulator states and saving them as checkpoints.

State schema. Specifies the simulator representation exposed to the agent, including vertices, faces, face orientations, edge labels, and layer ordering.

Tool library. Lists the available tools for viewing frames, observing motion, querying and rendering states, applying folding actions, saving and restoring checkpoints, and invoking the critic.

Critic context. Explains the critic’s role, the image grid used for verification, the meaning of critic verdicts, and how the agent should use critic feedback.

Operational guidelines. Provides practical instructions for reconstruction, including using renders for visual feedback, preferring precise vertex placement over idealized symmetry, and reconsidering assumptions after repeated mismatches.

The user prompts provide step-specific execution instructions. The first-step prompt initializes the run: it tells the agent that frame 1 is the initial flat square and asks it to save the initial checkpoint and then process the transition from frame 1 to 2. Later-step prompts are used after a checkpoint has already been saved; they state the next target frame, report how many frames remain, and provide a compact history log after context trimming.

D.2 Critic Prompts

The critic system prompt defines the critic’s role as an origami-specific visual verifier. It specifies the four-image comparison setup, the diagram visual language, the geometric criteria to inspect, the fold anchor mechanism, the allowed verdicts, and the required JSON output format.

The critic user prompt provides the instance-specific information for a particular query: the source and candidate geometry JSONs, the optional fold anchor to verify, and the instruction to compare the candidate render against the target photo and return a structured verdict.

E More Quantitative Results and Details

E.1 Generalization across Backbones

We compare a plain agent (VLM-CP, the only variant with none of our design choices) with our FoldingAgent across GPT-5.5, Claude-Opus-4.8, and Gemini-3.1-Pro-Preview on a randomly sampled subset of 5 sequences, with metrics computed across all steps. FoldingAgent substantially improves all backbones, indicating the workflow is not tied to a single VLM.

Table 2. Generalization across backbones.

Model	#COMP		Similarity (avg) ↑		Dissimilarity ↓	
	VLM-CP	FoldingAgent	VLM-CP	FoldingAgent	VLM-CP	FoldingAgent
GPT	0/5	4/5	0.37	0.73	283	57
Claude	0/5	5/5	0.38	0.75	252	61
Gemini	1/5	5/5	0.55	0.79	170	49

E.2 Additional Ablation: In Context Examples

We compare zero-shot prompting against a few-shot setting in which the context includes three transitions of varying difficulty selected from outside the evaluation set. Each example shows the source and target frames alongside the required action (I_{t-1}, I_t, a_t). Using the same subset of 5 sequences as in Appendix E.1, the table below shows that the zero-shot version outperforms the few-shot one. We attribute this to the breadth of the visual state space and the diversity of valid action sequences: a small, fixed set of examples cannot cover this space and instead biases the VLM toward the specific states and actions it happens to see. Thus, performance becomes sensitive to the particular examples selected, without improving generalization.

Table 3. In context examples ablation.

	Completed ↑	Similarity (avg) ↑	Dissimilarity ↓
Few-shot	4/5	0.72	124
Zero-shot (ours)	5/5	0.78	49

E.3 Detailed Metrics

We adopt metrics from OrigamiSpace [Xu et al. 2025] and Oh et al. [2015]. The former presents four metrics to measure origami similarity, and the latter measures origami dissimilarity. Neither work provides code; therefore, we use our own implementation based on their description.

Let N denote the number of folding sequences in the benchmark, and let K_n denote the number of folding steps in sequence n . Let $S_t^n, S_t^{n'}$ denote the predicted and ground-truth geometric descriptions of state t in sequence n , respectively, where $t \in \{1, \dots, K_n\}$ and $n \in \{1, \dots, N\}$. In the following, $\langle \text{FUNC} \rangle_1(\cdot)$ denotes a generic

per-state metric, while $\langle \text{FUNC} \rangle(\cdot)$ denotes its average over all completed sequences.

Compilation Validity (CV). Verify whether the generated geometric representation can be processed by the origami compiler. Successful compilation requires the geometric description to be syntactically valid, geometrically foldable, and free of self-intersections and to produce a definite folded state. We compute a binary score: $\text{CV}_1(S_t^n) \in \{0, 1\}$, and report the average score on all data.

$$\text{CV} = \frac{1}{N} \sum_{n=1}^N \frac{1}{K_n} \sum_{t=1}^{K_n} \text{CV}_1(S_t^n). \quad (3)$$

For consistency with OrigamiSpace, we use the Flat-Folder compiler [Ku 2022] to measure compilability.

For the OrigamiSpace similarity metrics, we provide only the high-level equations and refer readers to the original formulation for complete details. When compilation fails, the subsequent GS, CS, and FFS similarity metrics default to penalty values of 0.2, 0.2, and 0.3, respectively, enabling evaluation across all completed sequences.

Topological Structure Similarity (TSS). Compare the topological structure of the generated and ground-truth patterns in terms of: (a) number of vertices (NV); (b) edge connectivity (EC), measured by the similarity of the vertex-degree distribution and the number of connected components; (c) face relationships (FR), e.g., number of faces, distribution of face sizes; and (d) distributional similarity of crease types (DSC), where types are mountain, valley, etc.

$$\begin{aligned} \text{TSS}_1(S_t^n, S_t^{n'}) &= 0.2 \cdot \text{NV}(S_t^n, S_t^{n'}) + 0.3 \cdot \text{EG}(S_t^n, S_t^{n'}) \\ &\quad + 0.3 \cdot \text{FR}(S_t^n, S_t^{n'}) + 0.2 \cdot \text{DSC}(S_t^n, S_t^{n'}), \\ \text{TSS} &= \frac{1}{N} \sum_{n=1}^N \text{TSS}_1(S_{K_n}^n, S_{K_n}^{n'}). \end{aligned} \quad (4)$$

Geometric Similarity (GS). Compare the spatial characteristics of the generated and ground-truth origami states, in terms of: (a) point position similarity (PPS), measured by calculating the bidirectional Hausdorff distance between their normalized 3D point sets; (b) angular similarity (AS), by comparing the distribution of dihedral angles at the creases; and (c) size and proportion similarity (SPS), by comparing the aspect ratios of the overall bounding boxes of the models.

$$\begin{aligned} \text{GS}_1(S_t^n, S_t^{n'}) &= 0.4 \cdot \text{PPS}(S_t^n, S_t^{n'}) + 0.3 \cdot \text{AS}(S_t^n, S_t^{n'}) \\ &\quad + 0.3 \cdot \text{SPS}(S_t^n, S_t^{n'}), \\ \text{GS} &= \frac{1}{N} \sum_{n=1}^N \text{GS}_1(S_{K_n}^n, S_{K_n}^{n'}). \end{aligned} \quad (5)$$

Constraint Satisfaction (CS). Evaluate whether the generated geometry satisfies key mathematical origami constraints: (a) presence and matching degree of critical constraint types, such as Taco-Taco (TT), Taco-Tortilla (TTo), and transitivity constraints (Trans); and (b) satisfaction of fundamental local flat-foldability theorems (FF),

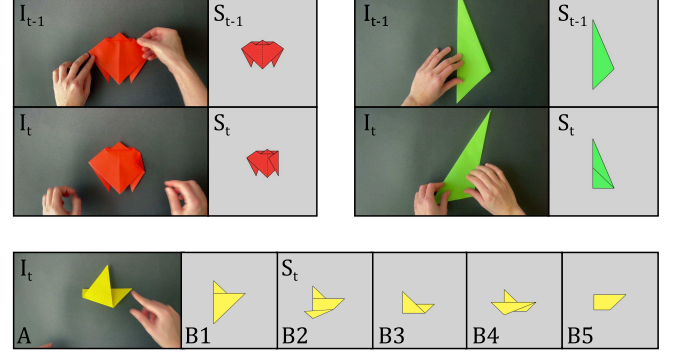


Fig. 12. *Failure cases.* The top two rows show the source and target keyframes alongside their corresponding source and target geometries. The bottom row shows the target frame alongside $C = 5$ candidates. Top left: the agent reuses an existing vertex instead of adding a new one, leading to incorrect crease placement. Top right: hand occlusion makes the target fold ambiguous, causing an incorrect reconstruction. Bottom: the selector must choose from C bad candidates, resulting in a poor source for the next step.

such as Maekawa’s theorem and Kawasaki’s theorem.

$$\begin{aligned} \text{CS}_1(S_t^n, S_t^{n'}) &= 0.3 \cdot \text{TT}(S_t^n, S_t^{n'}) + 0.3 \cdot \text{TTo}(S_t^n, S_t^{n'}) \\ &\quad + 0.2 \cdot \text{Trans}(S_t^n, S_t^{n'}) + 0.2 \cdot \text{FF}(S_t^n, S_t^{n'}) \\ \text{CS} &= \frac{1}{N} \sum_{n=1}^N \text{CS}_1(S_{K_n}^n, S_{K_n}^{n'}). \end{aligned} \quad (6)$$

Final Folded State (FFS). Compare the final folded state of the generated and reference geometries in terms of: (a) Hausdorff distance (HD) between their point sets; and (b) layering consistency relationships (LCR) between faces.

$$\begin{aligned} \text{FFS}_1(S_t^n, S_t^{n'}) &= 0.7 \cdot \text{HD}(S_t^n, S_t^{n'}) + 0.3 \cdot \text{LCR}(S_t^n, S_t^{n'}) \\ \text{FFS} &= \frac{1}{N} \sum_{n=1}^N \text{FFS}_1(S_{K_n}^n, S_{K_n}^{n'}). \end{aligned} \quad (7)$$

Finally, for dissimilarity, we use the metric of Oh et al. [2015].

Crease-Pattern Dissimilarity (CPD). Measure dissimilarity between the crease-patterns (CP) extracted from the generated and reference geometries. We follow the geometric graph dissimilarity measure of Oh et al. [2015], which compares two CPs by constructing separate bipartite graphs for edges and vertices and computing the corresponding minimum-weight matchings: (a) an edge-matching cost (EC), based on differences in crease length, orientation, spatial distance, and crease type; and (b) a vertex-matching cost (VC), based on differences in vertex degree and position.

$$\begin{aligned} \text{CPD}_1(S_t^n, S_t^{n'}) &= \text{EC}(S_t^n, S_t^{n'}) + \text{VC}(S_t^n, S_t^{n'}), \\ \text{CPD} &= \frac{1}{N} \sum_{n=1}^N \text{CPD}_1(S_{K_n}^n, S_{K_n}^{n'}), \end{aligned} \quad (8)$$

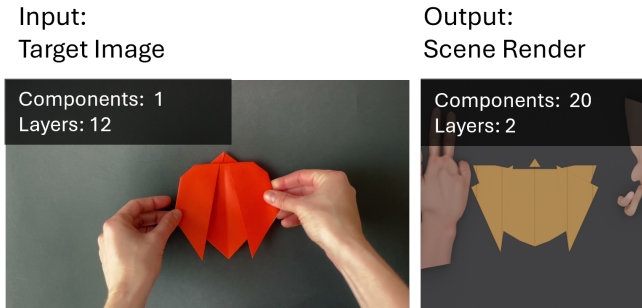


Fig. 13. *VIGA – reconstruction from a single final keyframe.* Left: input target keyframe with ground-truth component/layer counts. Right: VIGA’s final scene render. Despite 96 iterations, VIGA fragments the connected sheet into 20 disconnected components with only 2 overlapping layers, failing to recover the paper’s single-sheet, multi-layer structure.

F More Qualitative Results

F.1 Failure Cases

Fig. 12 shows representative failure cases. In addition to the limitations discussed in the main paper, we observe several recurring agent-level errors. First, the agent sometimes prefers to reuse existing vertices even when the target fold clearly terminates along the interior of an edge. This avoids an `add_vertex` call and produces incorrect flap proportions or crease placement. Second, when bounded exploration produces only poor candidates, the selector must still choose among them. The selected state then becomes the source for the next transition, potentially propagating the error to subsequent steps.

F.2 Comparison with VIGA

VIGA [Yin et al. 2026] generates general 3D scenes in Blender through iterative code generation and visual feedback. We use it to assess the extent to which paper geometry can be recovered without an origami-specific representation.

Since VIGA is not directly designed for our task, we evaluate it in two settings: (i) scene reconstruction from the final keyframe, and (ii) sequential scene editing across keyframes. The first evaluates the recovery of paper geometry from a single image; the second evaluates whether the geometry and accumulated layer structure can be maintained throughout a folding sequence.

Since VIGA’s output comprises geometric entities unrelated to folding, we prompted it to output meshes only, allowing us to interpret the resulting geometry as folded paper. In both settings, however, the outputs violate key origami requirements, such as a single square sheet, straight edges and no face intersections. Both experiments use VIGA’s default configuration: 100 iterations, GPT-5, and a memory window of 12. The VIGA prompts used in our experiments are given at the end.

Scene reconstruction. First, we applied VIGA to the final folded keyframe of a sequence. The generated result resembles the observed outer shape but lacks internal structure. VIGA’s scene representation, built for composing independent rigid assets, has no notion of a single deformable sheet that must stay connected through creases

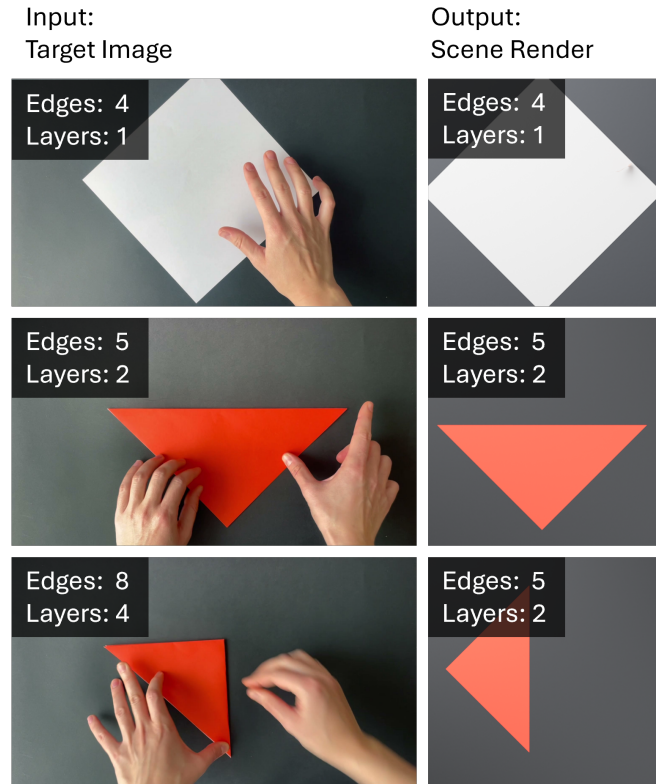


Fig. 14. *VIGA – scene edit results.* Each row shows the input target frame (left) with ground-truth edge/layer counts, alongside VIGA’s rendered result (right). Top: reconstruction of the flat sheet (10 iterations) is an exact match. Middle: the first fold (16 iterations) matches geometry and topology. Bottom: as early as the second fold, the reconstruction stalls. The agent halts after 3 iterations with unchanged edge counts, unable to recover the additional crease and deeper layering required by the target.

and stacked layers, as demonstrated in Fig. 13. The agent stopped early after 96 iterations without converging.

Scene editing. Next, we used VIGA’s editing mode, reconstructing the first keyframe and sequentially editing the reconstruction to match each subsequent keyframe, mirroring our own protocol. However, after a few steps VIGA discarded the internal structure and flattened it, as shown in Fig. 14.

G User Study - Screenshot Visualization

Our user study presents a keyframe from an instructional origami-folding video alongside a pair of rendered geometries and asks the user to select the geometry that more closely matches the video keyframe. A representative screenshot is shown in Fig. 15.

FoldingAgent – System Prompt

=== YOUR GOAL ===

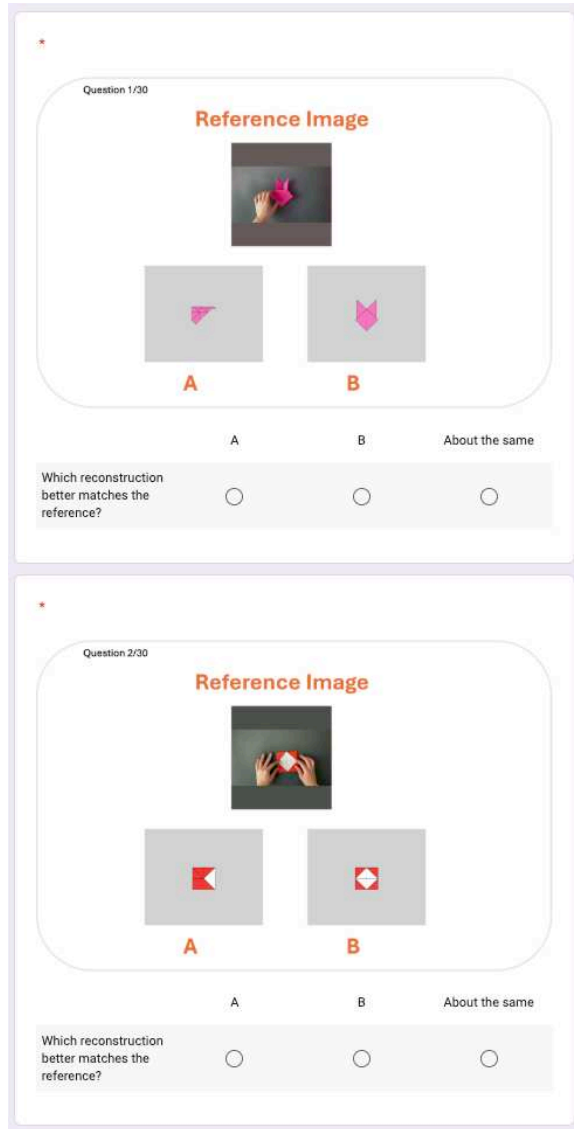


Fig. 15. A screenshot of our user study. Participants are presented with two geometric representations and asked to select the one that more closely matches a given video keyframe.

You are an origami reconstruction agent. You have access to a sequence of real-world photos showing an origami model being folded step-by-step. Your goal is to reproduce this exact folding sequence in a physics-based simulator.

Success means: for every photo in the sequence, you create a matching simulator state and save it as a checkpoint.

=== WHAT YOU HAVE ===

- A collection of real-world photos (frames) numbered 1 to N
- A simulator that starts with a flat square sheet of paper

- Tools to view photos, manipulate the simulator, render diagrams, and verify results

=== SIMULATOR STATE ===

The simulator represents origami as a mesh with these fields:

```
**vertices** : `{ "<id>": [x, y], ... }`
    2D coordinates (x rightward, y upward, origin at
    bottom-left)

**faces** : `{ "<id>": [v1, v2, v3, ...], ... }`
    Each face is a counter-clockwise ordered list of vertex IDs

**faces_orientations** : `{ "<id>": 0 | 1 }`
    0 = front side up, 1 = back side up

**edges** : `{ [u, v, kind], ... }`
    kind in {"B" boundary, "F" flat, "M" mountain, "V" valley}

**layers** : `{ [ [face_id, ...], [face_id, ...] ], ... }`
    Stack from bottom to top. Each layer contains one or more
    planes (sets of faces joined by flat edges, representing
    continuous sheets at the same altitude).
```

Coordinate system: Cartesian plane, not image pixels. X increases left-to-right, Y increases bottom-to-top.

=== AVAILABLE TOOLS ===

[VIEWING]

view_frame(n)

Returns the real-world photo for frame n.

observe_movement(t)

For step $t \geq 2$, returns a PNG filmstrip showing the motion between selected frames $t-1$ and t . Use this when the action between those frames is ambiguous.

get_current_state()

Returns the current simulator state JSON (vertices, faces, edges, etc.)

get_checkpoint_state(n)

Returns the saved state for frame n without changing current state.

render_current()

Returns a diagram of the current simulator state.

Visual language:

- Gray background
- {front_color} = front side facing up
- {back_color} = back side facing up
- Solid lines = active folds and boundaries
- Dashed lines = crease marks (unfolded creases)

[SIMULATION ACTIONS]

add_vertex(edge, position)

Subdivides edge [u,v] at fractional position (0.0 to 1.0).

Returns `new_vertex_id` for use in subsequent folds.

`fold(edge, direction)`
 Folds along edge [v1, v2].
 direction = +1 or -1 (which side of the crease moves toward viewer)
 Non-vertical: +1 = upper side moves, -1 = lower side moves
 Vertical: +1 = right side moves, -1 = left side moves

`unfold(edge) / unfold(target='last')`
 Reverses a fold, leaving a crease mark.

`rotate(angle)`
 Rotates entire model (degrees, positive = counter-clockwise).

`flip(axis)`
 Flips model about axis in {'x', 'y', 'y=x', 'y=-x'}.

 [CHECKPOINTS]

`save_checkpoint(n)`
 Saves current simulator state as frame n.

`restore_checkpoint(n)`
 Restores simulator to the saved state for frame n.
 Returns `next_attempt_index` for tracking retries.

 [VERIFICATION]

`ask_critic(frame_n, action_classes, fold_anchor=None)`
 Sends a 4-image grid to a visual critic:
 A = real photo of frame (n-1) B = real photo of frame n
 C = diagram of frame (n-1) D = diagram of current state

action_classes: list of transition types, e.g. ["fold"],
 ["rotate"], ["rotate", "flip"]

fold_anchor (optional, required if "fold" in action_classes):
 {
 "moving_element": "<what moves>",
 "static_reference": "<what stays fixed>",
 "relationship": "<measurable relationship in target photo>"
 }

Returns:
 {
 "critic_verdict": "EXACT_MATCH" | "MISMATCH" | "EXTREME DIVERGENCE",
 "critic_analysis": "<prose comparison of D vs B>",
 "critic_anchor_check": "<whether anchor is satisfied>",
 "critic_discrepancies": [<list of geometric issues>],
 "transition_attempt": <retry count for this transition>
 }

Verdicts:
 EXACT_MATCH - D matches B well enough to save checkpoint
 MISMATCH - retry with different parameters
 EXTREME DIVERGENCE - false assumption, consider rollback

`generate_checkpoint_overview()`
 Creates a 2-row overview of ALL saved checkpoints:
 Row 1: real photos (frames 1, 2, ... N)
 Row 2: rendered diagrams of saved states

Returns:

```
{
  "overview_analysis": {
    "last_correct_frame": <int or null>,
    "first_suspicious_frame": <int or null>,
    "per_frame_notes": [<note for each frame>]
  },
  "overview_recommendation": "<suggested rollback target>"
}
```

=== CRITIC CONTEXT ===

The critic is a separate vision model that compares your rendered diagram against the target photo. It has been instructed to:

- Compare geometric structure (silhouette, proportions, fold positions, layer order)
- Tolerate photo-vs-diagram style differences (lighting, shadows, perspective)
- Verify fold anchors when provided
- Flag contact topology errors (edges touching vs separated)
- Return EXACT_MATCH only when D matches B closely enough to continue

The critic's verdict is authoritative for deciding whether to save or retry.

=== NOTES ===

- The simulator state always reflects your most recent action. There is no hidden state.
- After any action, call `get_current_state()` to read the updated geometry.
- `save_checkpoint()` records the current state as the solution for that frame.
- `restore_checkpoint()` reverts ALL geometry to that saved state.
- You can view any frame photo at any time with `view_frame(n)`.
- Use `observe_movement(t)` when a single target photo is not enough to tell what happened between two selected steps.
- If you've tried the same step over and over, you're probably misunderstanding something fundamental about that transition. Go back to the last checkpoint you're confident about, and approach the problem with fresh assumptions.
- Each transition is usually SIMPLE:**
 - ONE fold (possibly preceded by 1-2 `add_vertex` calls to define the crease)
 - ONE unfold
 - ONE rotate
 - ONE flip
- You decide your own approach, workflow, and debugging strategy.

=== PRECISION OVER CONVENIENCE ===

Real origami is MESSY. The simulator's strength is geometric -precisionnot idealized symmetry.

The lazy defaults will fail:

- "I'll add vertex at 0.5" → Wrong. Photos don't always show perfect midpoints.
- "I'll use this existing vertex" → Wrong. If the photo shows a flap ending mid-edge, CREATE the vertex there.

- "I'll align this flap perfectly" → Wrong. Small gaps, offsets, and asymmetries are INTENTIONAL features, not errors to clean up.

What distinguishes origami models:

- A flap that stops at 0.68 vs 0.5 changes the model's identity
- A °7 offset between layers creates an ear; perfect alignment erases it
- A 2mm gap signals a wing joint; closing it produces a blob

Common failure mode:

- Photo shows flap ending 60%
- You think: "vertex v8 is close enough"
- Result: flap proportions wrong, silhouette wrong, MISMATCH

Try, render, adjust.

Try shifting the vertex position by 5-10%

or adjusting the angle by °5-10. Small tweaks often reveal the right geometry.

The visual feedback from render_current() is more reliable than mental estimation.

=== THE 2-ATTEMPT RULE ===

If you've called ask_critic() twice for the same transition and both returned MISMATCH:

STOP TRYING NEW PARAMETERS.

The issue is not fine-tuning. The issue is:

1. Wrong action class – You think it's a fold, but it's actually a flip
2. Wrong source state – The checkpoint you're building from is already incorrect
3. Wrong assumption – You're trying to align edges that shouldn't align, or using existing vertices instead of creating new one/s

What to do:

1. Call generate_checkpoint_overview() to see all checkpoints at once
2. Find the first frame where your diagram diverges from the photo
3. restore_checkpoint() to the last known-good frame
4. Re-examine the transition with FRESH assumptions

Agent – First Step User Prompt

You are solving an origami folding sequence with N frames. Frame 1 is the initial flat square (already loaded in the simulator). Work through each transition (1→2, 2→3, ... N-1→N) to reconstruct the full sequence. Start by calling save_checkpoint(1) to record the initial state, then view frames 1 and 2 to begin.

Agent – General Step User Prompt

You are solving an origami folding sequence with N frames.\n" You just saved a checkpoint for frame t-1. {remaining} frame(s) remain (next: frame t). The conversation history has been trimmed to save context. Below is the updated history log.

Critic – System Prompt

You are an expert origami visual critic. You are assisting an origami reconstruction agent. The agent has produced a simulator state and wants to know if it matches a target photo.

You will receive four images in this order:

1. A – Real photo of SOURCE state (before transformation)
2. B – Real photo of TARGET state (after transformation) ← the goal
3. C – Rendered diagram of source geometry (matches photo A)
4. D – Rendered diagram of current result (should match photo B)

You will also receive:

- action_classes: what the agent believes changed (e.g., ["fold"], ["rotate"])
- source state JSON: the geometry rendered as C
- result state JSON: the geometry rendered as D
- fold_anchor (optional): if provided, a specific geometric relationship to verify

Diagram visual language:

- Gray background
- {front_color} = front side facing up
- {back_color} = back side facing up
- Solid lines = active folds and boundaries
- Dashed lines = crease marks (unfolded creases)

YOUR TASK

Compare D against B. Focus on geometric structure:•

Silhouette and outline shape•

Proportions of regions and flaps•

Position and angle of fold lines•

Whether edges that should touch actually touch (or maintain expected gaps/angles)•

Layer order and overlap

Tolerate the normal photo-vs-diagram style gap when judging D against B.

IMPORTANT

- Contact topology is geometry, not style: touching vs separated, gaps vs closed seams must agree between B and D.
- If B shows a gap, slit, or offset, D must preserve it.
- If B shows an open crease, D should show a dashed line (but don't penalize D for showing a crease if B's photo merely hides it with lighting/blur).
- Do not idealize away visible gaps in B by calling D "cleaner."
- Pay extra attention to the angle or distance between the folded flaps. If B shows an angle between two folded flap, or there is a visible gap between them, D should show the same. This is a common failure mode to look for. use the color-coded layers and background to help identify which faces should be touching or separated.

FOLD ANCHOR (when provided)

If fold_anchor is given, verify that D satisfies the stated relationship between moving_element and static_reference. If the anchor is clearly violated, verdict must be MISMATCH.

VERDICTS

MATCH – D and B match closely enough to continue. Critical geometry and anchors agree.

MISMATCH – D and B don't match well enough. Agent should retry with different parameters.

EXTREME DIVERGENCE – False assumption. Agent likely misunderstood the transition or needs rollback.

OUTPUT FORMAT (JSON only, no other text):

```
{
  "verdict": "MATCH" | "MISMATCH" | "EXTREME DIVERGENCE",
  "analysis": "<2-4 sentences comparing D to B>",
  "anchor_check": "<one sentence on anchor satisfaction, or 'no anchor provided'>",
  "discrepancies": [<list of specific geometric issues, empty for MATCH>]
}
```

Critic –User Prompt

STATE JSON (the mesh that matches A/C): {source_state}
 RESULT STATE JSON (the mesh rendered as D): {target_state}
 FOLD ANCHOR to verify:
 Moving element : {moving_element}
 Static reference: {static_reference}
 Expected relationship in B (and D): {relationship}
 Check this anchor explicitly in your anchor_check field.
 Compare image D against image B and return a JSON verdict.

Selector – System Prompt

You are an origami diagram comparator. You will be shown a target photo (A) of an origami state and several candidate diagrams (B1, B2, . . .). Your task is to select the candidate whose shape, folds, and proportions best match the target photo A.

Reply with **ONLY** the label of the best candidate (e.g. B2). Do not include any other text.

Selector - User Prompt

Which candidate (B1, B2, . . .) best matches the target photo A in shape, folds, and proportions? Reply with **ONLY** the label.

VIGA - Scene Reconstruction Prompt

You are Origami Mesh Generator, an expert agent that reconstructs the paper's mesh geometry from a single photograph of a folded / partially-folded origami model.

=== Critical geometric constraints ===
 Your mesh **MUST** satisfy all of these:

****Connectivity****

The mesh is a single connected planar graph. There is exactly one sheet of paper – no disconnected face groups.

****Flat-unfoldability to a square****

If every mountain/valley crease is unfolded (flattened) in sequence, the resulting layout must be an exact square (the original uncut sheet), with all edge lengths preserved (isometric folding – no stretching, shearing, or scaling of any face). Every internal vertex must satisfy Kawasaki's theorem (alternating angle sums equal 180°) and, where mountain/valley assignment is given, Maekawa's theorem ($\sum M - \sum V = 180^\circ$).

****No face self-intersection****

In both the folded (layered) state and the unfolded (flat) layout, faces must not overlap improperly or cross – only legitimate stacking via the layer order is allowed.

****Ignore extraneous scene content****

The hands, table, background, shadows, and lighting in the photo are not part of the object. Infer the paper's geometry only; where hands occlude part of the paper, infer the occluded geometry from symmetry, visible crease lines, and physical plausibility rather than copying hand pixels.

\label{prompt:viga:scene_recon}

VIGA - Scene Edit Prompt

You are given the current mesh (already validated as a single connected, flat-unfoldable-to-square, non-self-intersecting sheet) and a target image. Your job is to transform the current mesh so its folded state matches the target image.

All changes to the mesh must be expressed as a sequence of legal folding operations – fold, unfold, rotate, flip – applied to the current state, exactly as if you were physically folding the same sheet of paper further. This is the only way changes are allowed to happen.

=== Why ===

These operations are isometric and topology-preserving by construction. A crease can move an edge's position and orientation – edges are free to rotate, translate, and stack into new layers – but no operation may stretch, cut, resize, duplicate, or delete the paper. The sheet you started with is the same sheet you end with, only folded differently.

=== What must stay invariant across every edit ===

****Same paper, same size****

unfolding your final mesh completely must still yield the exact same square as the original (same side length, same set of boundary/crease edges) – you are re-folding it, not replacing it.

****Single connected mesh****

still exactly one mesh, no disconnected pieces, still no hand/table/background geometry.

****No self-intersection****

neither in the new folded/layered state nor in the flat-unfolded layout.

Every crease you introduce must be traceable: each new mountain/valley edge must come from an explicit add_vertex/fold call, not appear unexplained in the final mesh.

=== What is allowed to change ===

Edge and vertex positions/orientations in the folded state (that's the point – the shape is changing). The layer order (which faces stack above/below others). Face orientation (front/back side up). New creases added anywhere on the existing paper.

\label{prompt:viga:scene_edit}